# 第七章

# 图

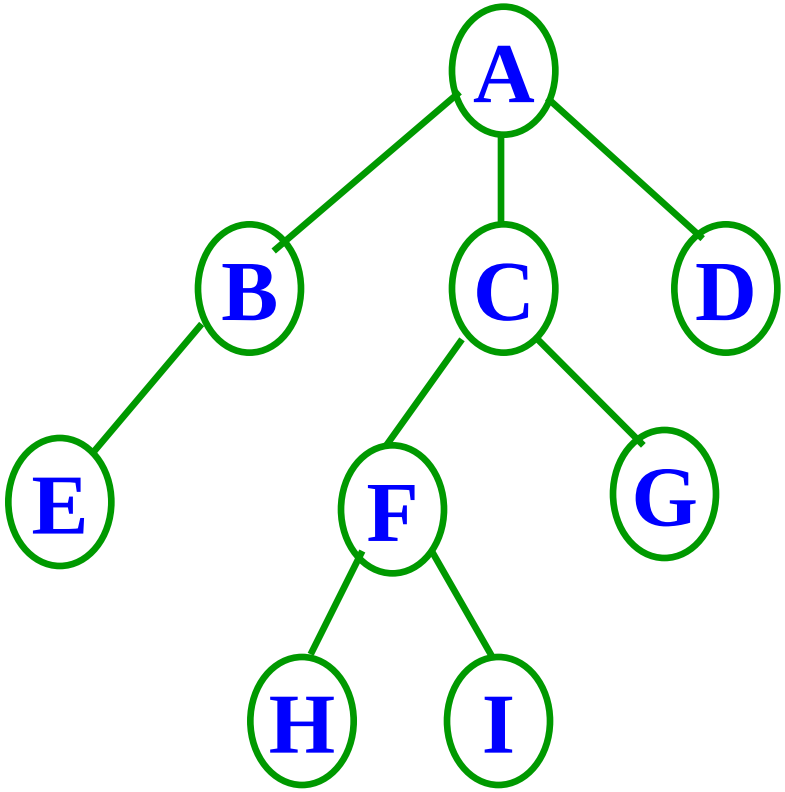
## 7.1 图的基本概念

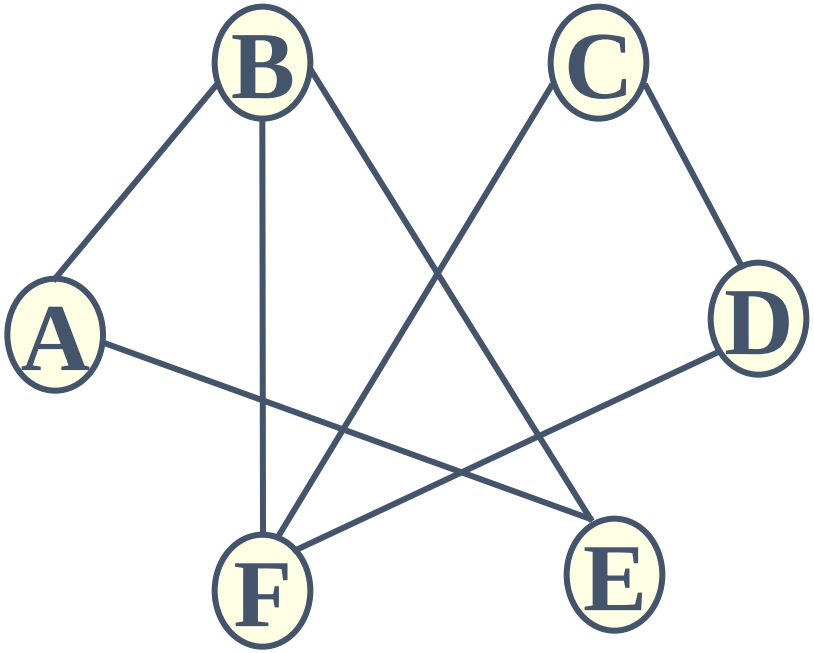
## 7.2 图的存储结构

## 7.3 图的遍历

## 7.4 最小生成树

$(a_1, a_2, \ldots a_{i-1}, a_i, a_{i+1}, \ldots, a_n)$



# 三种结构比较

**线性表：**数据元素之间仅有线性关系，即每个数据元素只有一个直接前驱和一个直接后继。

**树：**数据元素之间有着明显的层次关系，虽然每一层上的数据元素可能和下一层中多个元素(孩子)相关，但只能和上一层中一个元素(双亲)相关。

**图：**一种较线性表和树更为复杂的数据结构。在图结构中，结点之间的关系可以是任意的，任意两个数据元素之间都可能相关。

# 7.1 图的基本概念

## 一、图的结构定义

图是由一个顶点集 V 和一个边集 E构成的数据结构。

二元组表示： G = (V, E) ，其中，V是顶点的有穷非空集合，E是V中顶点偶对的有穷集。

$$E = \{<v,w> \mid v,w \in V \text{ 且 } P(v,w)\}$$

<v,w>表示从 v 到 w 的一条弧，并称 v 为弧尾，w 为弧头。

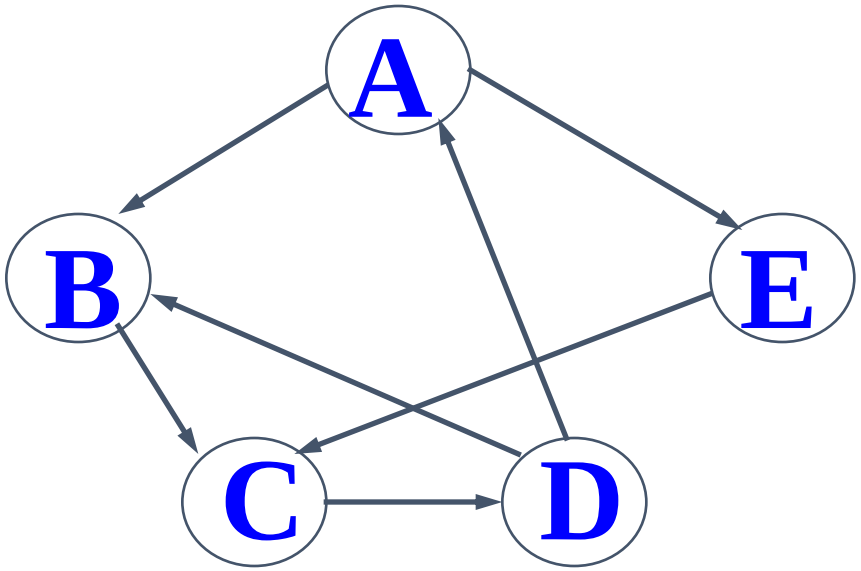
谓词 P(v,w) 定义了弧 <v,w>的意义或信息。

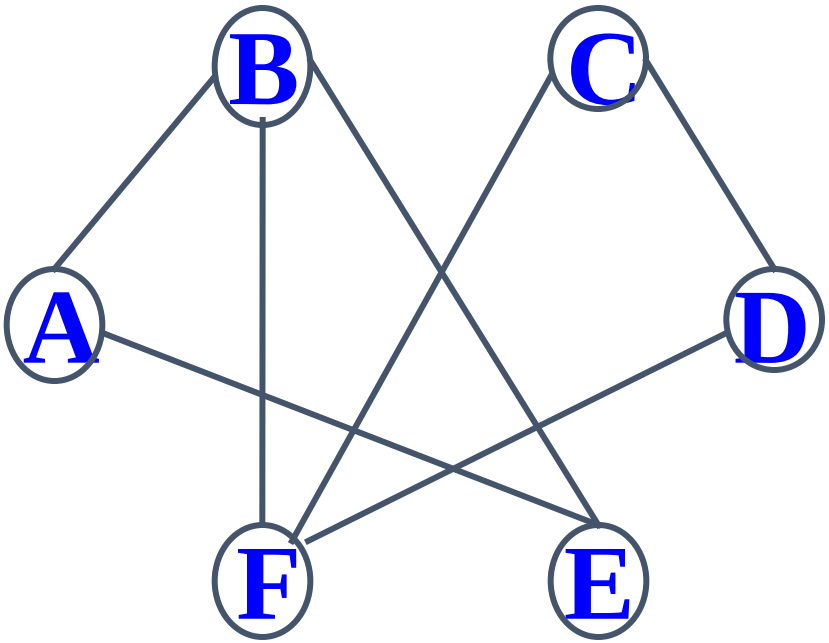
# 1）顶点、边、弧

顶点：图中的数据元素。

边：若<v，w>∈E 必有<w，v>∈E，则称（v,w）为顶点 v 和顶点 w 之间存在一条边。

弧：若<v，w>∈E 是有方向的,则称 <v，w>为顶点 v 和顶点 w 之间存在一条有向边，也称为弧。



# 2）无向图、有向图

无向图：由顶点集和边集构成的图。

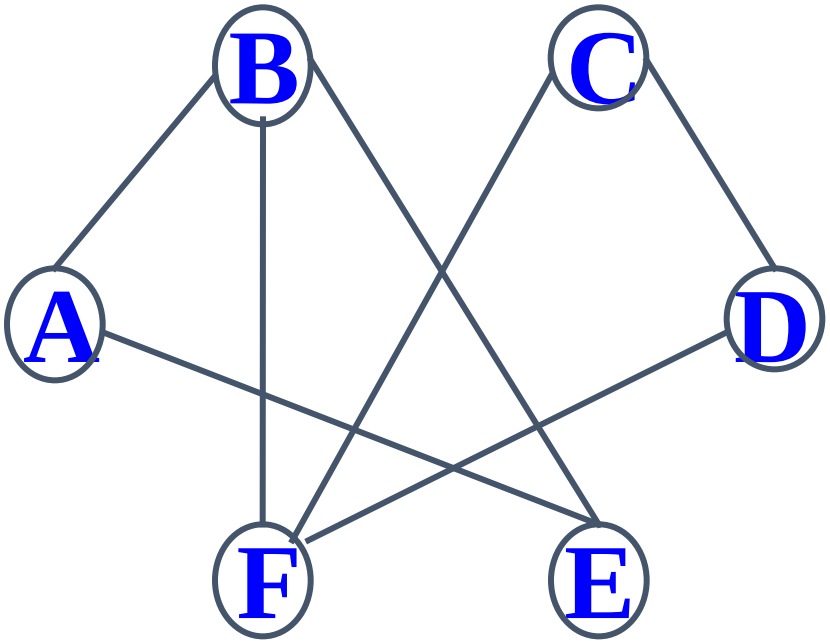
例如：$G_2=(V_2, E_2)$

$V_2=\{A, B, C, D, E, F\}$

$E_2=\{(A,B), (A,E),$

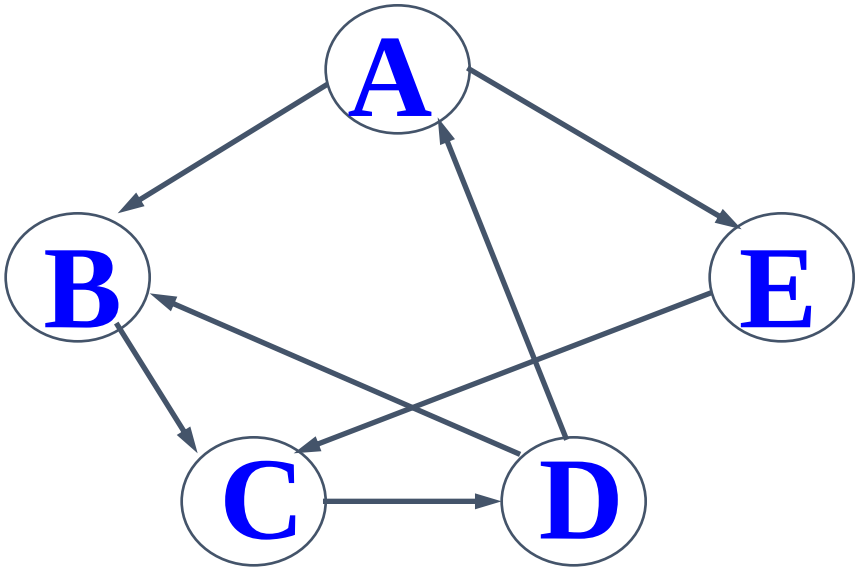
$(B,E), (C,D), (D,F),$

$(B,F), (C,F)\}$



有向图：由于“弧”是有方向的，因此称由顶点集和弧集构成的图为有向图。

例如： $G_1 = (V_1, E_1)$




$V_1$={A, B, C, D, E}

$E_1$={<A,B>, <A,E>,
<B,C>, <C,D>, <D,B>,
<D,A>, <E,C> }

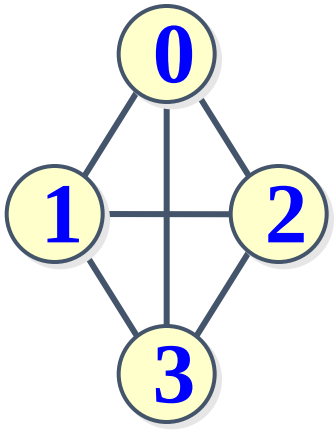
# 3）完全图、稀疏图、稠密图

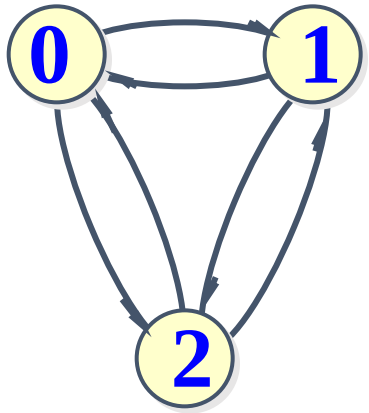
假设图中有 n 个顶点，e 条边，则

含有 e=n(n-1)/2 条边的无向图称作无向完全图；

含有 e=n(n-1) 条弧的有向图称作 有向完全图；

无向完全图




有向完全图

若边或弧的个数e<nlogn，

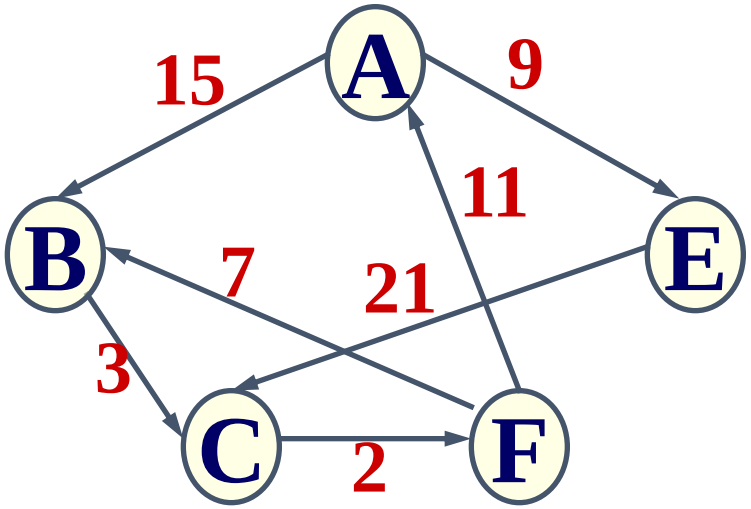
则称作稀疏图，否则称作稠密图。

## 4）权、网

权：某些图的边具有与它相关的数，称之为权。

权可以代表一个顶点到另一个顶点的距离，耗费等。这种带权图叫做网络。

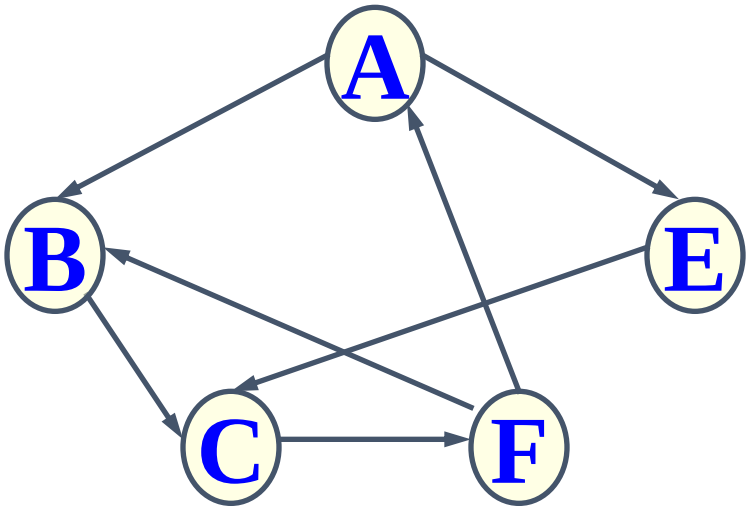
弧或边带权的图分别称作有向网或无向网。

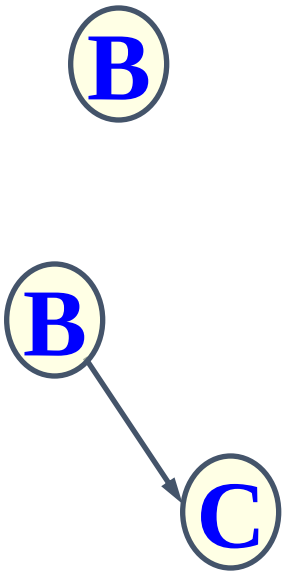


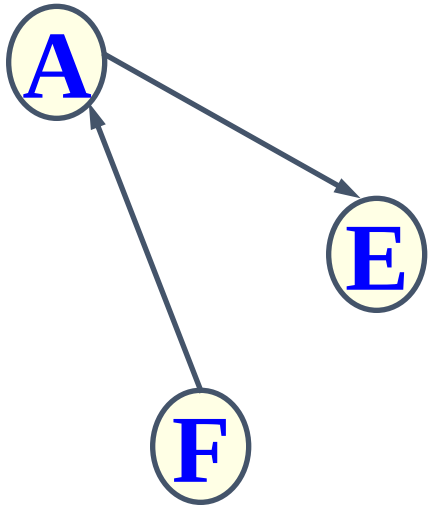
## 5）子图

设图G=(V,E) 和图 G′=(V′,{E′}),

且 V′⊆V, E′⊆E,则称 G′ 为 G 的子图。



# 6）邻接点、度、入度、出度

假若顶点v 和顶点w 之间存在一条边，

则称顶点v和w互为邻接点。

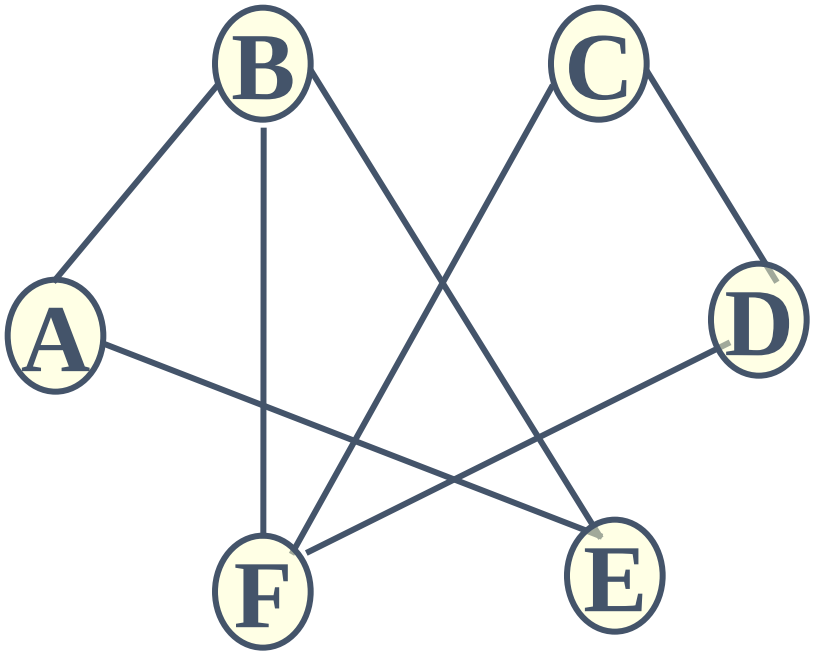
边(v,w) 和 顶点v与w相关联。

和顶点v关联的边的数目定义为顶点v的度。

例如：

ID(B) = 3

ID(A) = 2



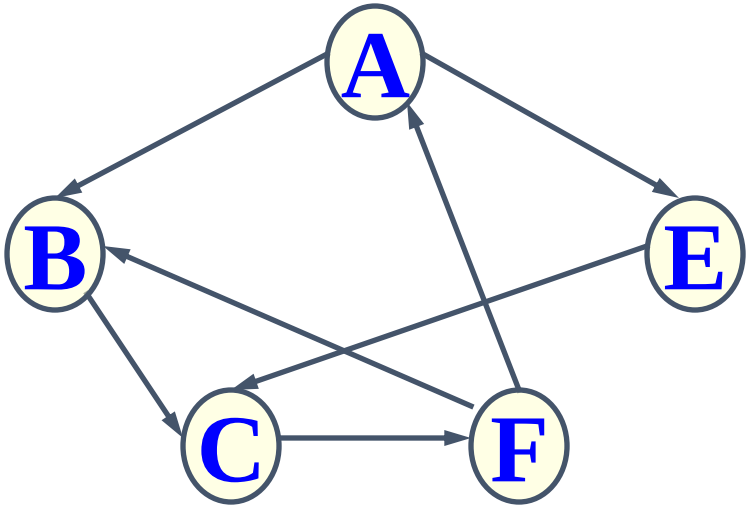
对有向图来说，

出度：以顶点v为弧尾的弧的数目。

入度：以顶点v为弧头的弧的数目。

顶点的度(TD)=

出度(OD)+入度(ID)




例如：

OD(B) = 1

ID(B) = 2

TD(B) = 3

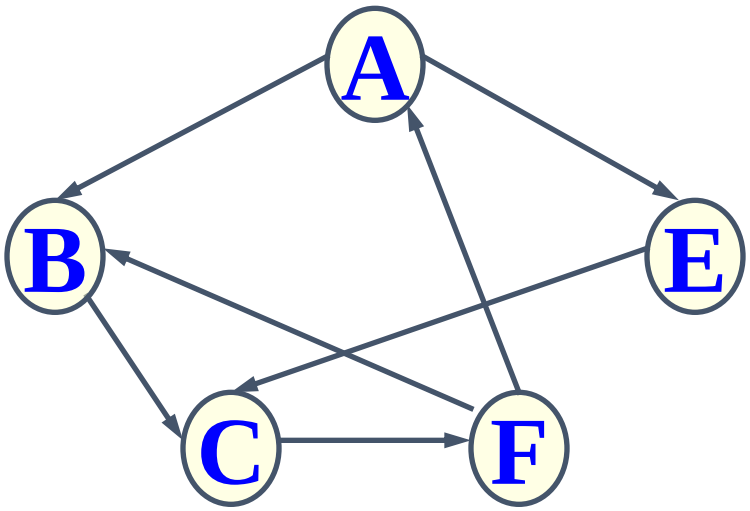
# 7） 路径

设图G=(V,E)中的一个顶点序列

$\{u=v_{i,0},v_{i,1},\cdots,v_{i,m}=w\}$中，$(v_{i,j-1},v_{i,j})\in E$ $1\le j\le m$,

则称从顶点u 到顶点w 之间存在一条路径。

路径上边的数目称作路径长度。



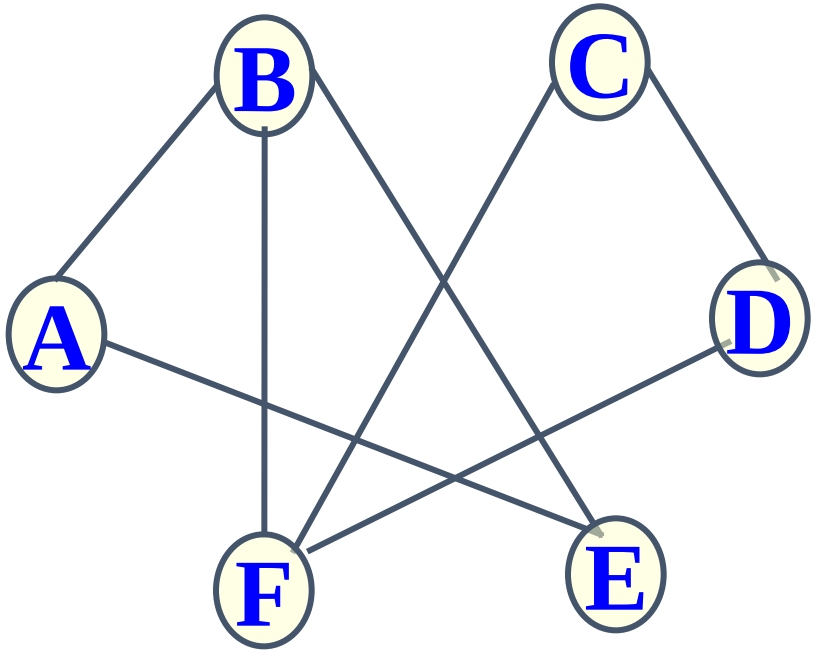

从A到F长度为3 的路径{A,B,C,F}

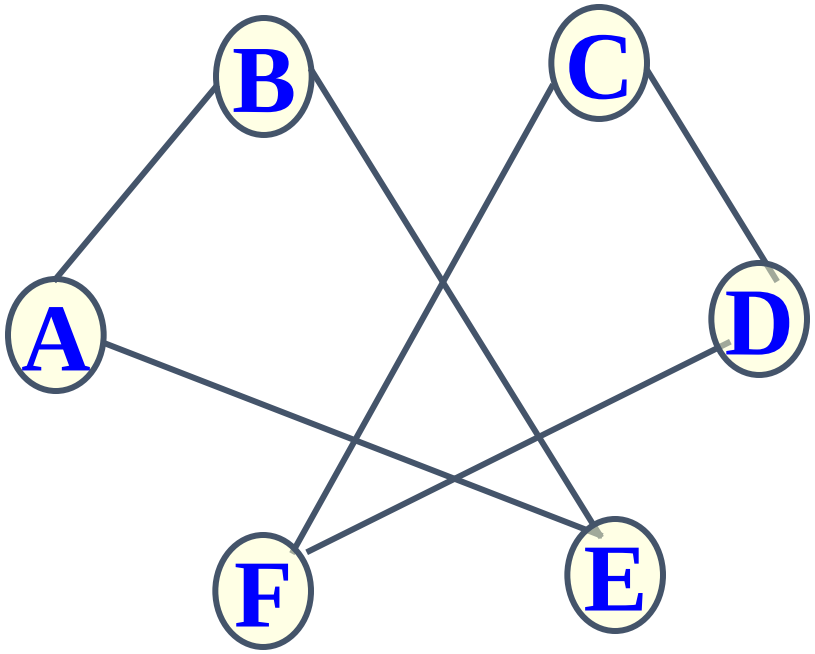
简单路径：指序列中顶点不重复出现的路径。

简单回路：指序列中第一个顶点和最后一个顶点相同的路径。

# 8） 连通图、连通分量

若图G中任意两个顶点之间都有路径相通，则称此图为连通图。

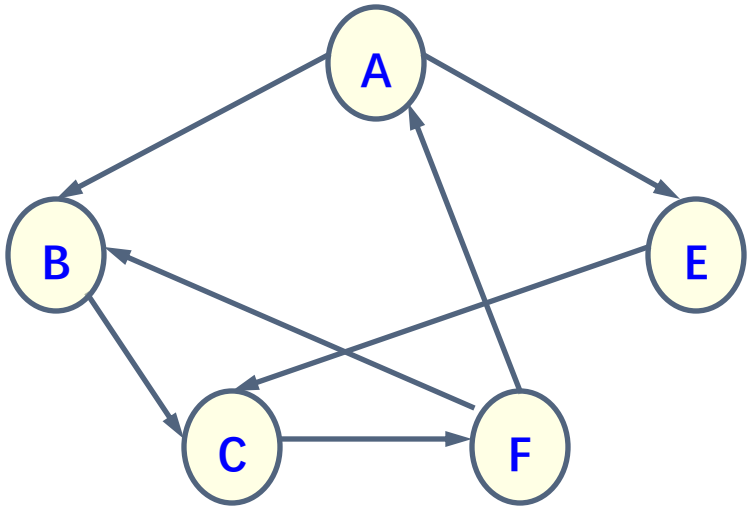


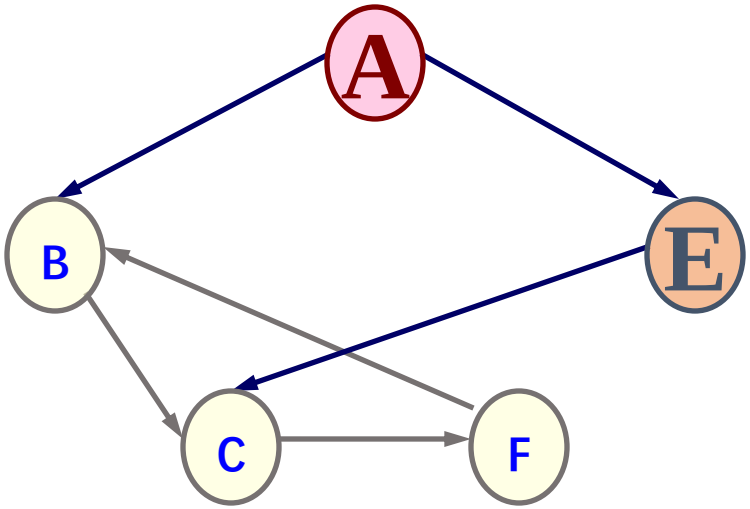




若无向图为非连通图，则图中各个极大连通子图称作此图的连通分量。

对有向图，若任意两个顶点之间都存在一条有向路径，则称此有向图为强连通图.

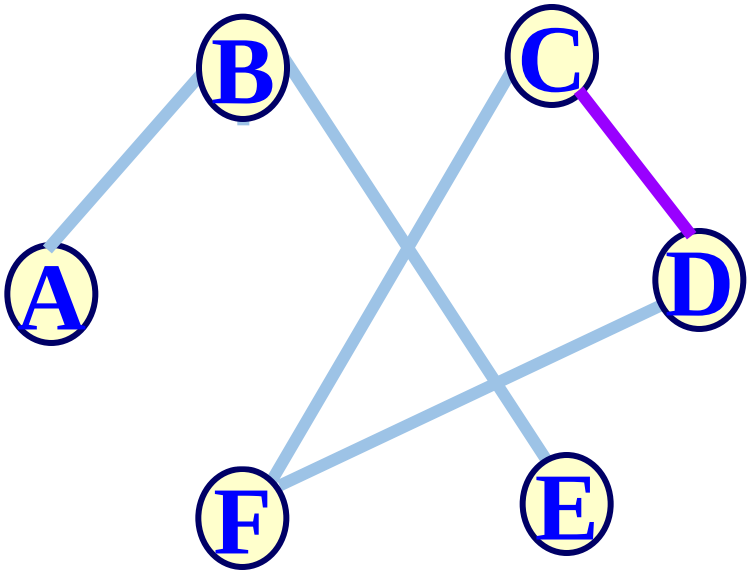
否则，其各个强连通子图称作它的强连通分量。



# 9） 生成树、生成森林

假设一个连通图有 n 个顶点和 e 条边，其中 n-1 条边和 n 个顶点构成一个极小连通子图，称该极小连通子图为此连通图的生成树。

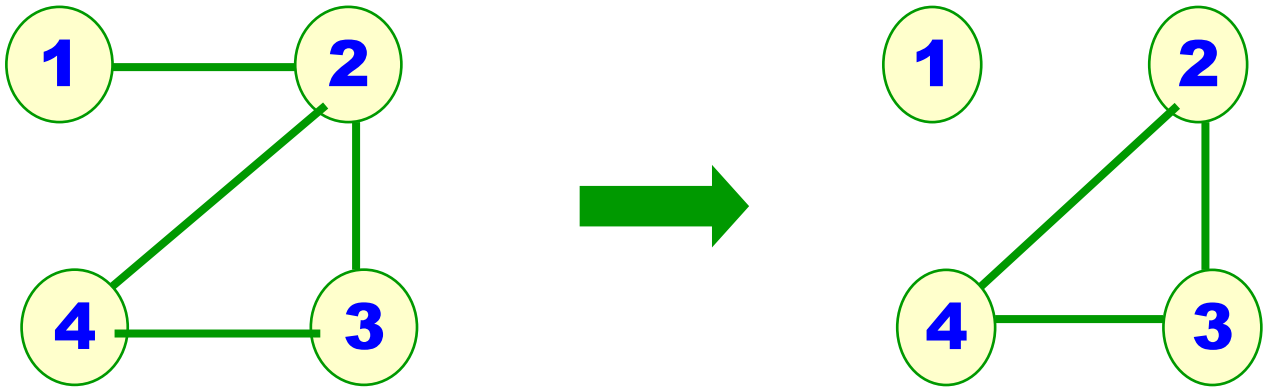



在极小连通子图中增加一条边，则一定有环。

在极小连通子图中去掉一条边，则成为非连通图。

## 注意：

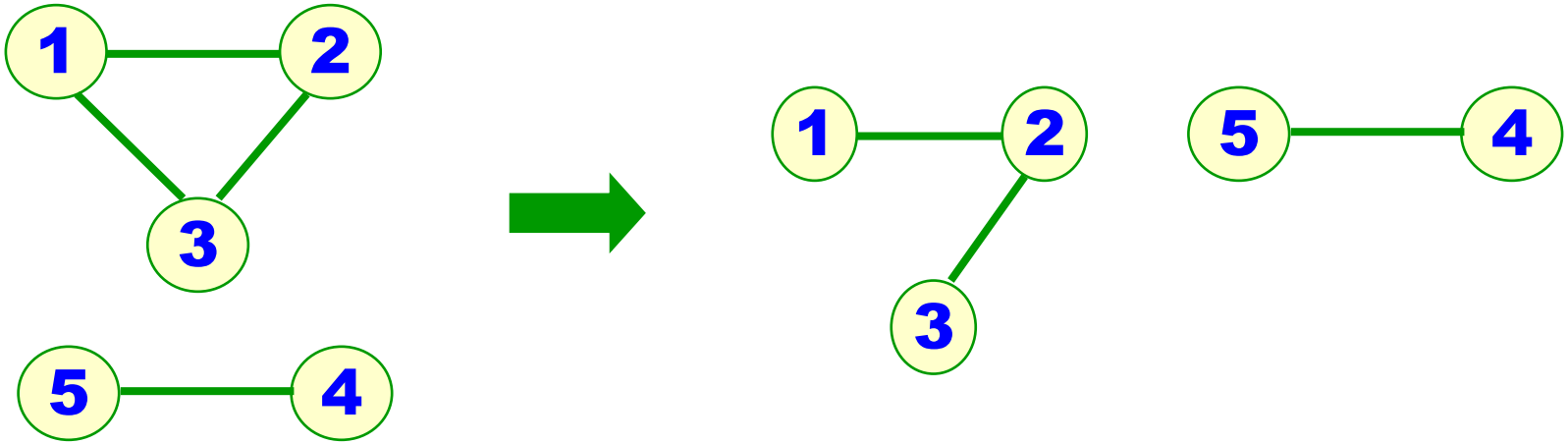
有n个顶点，n-1条边的图不一定是生成树



# 非连通图的生成树：

在非连通图中，由每个连通分量都可得到一个极小连通子图，即一棵生成树。

这些连通分量的生成树就组成了一个生成森林。




若图中有n个顶点，m个连通分量，

则生成森林中有n-m条边。

# 二、抽象数据类型图的定义

ADT Graph{

数据对象V：V是具有相同特性的数据元素的集合，称为顶点集。

数据关系R：R={VR}

VR={<v,w>| v,w∈v且P(v,w)，<v,w>表示从v 到w 的弧，谓词P(v,w)定义了弧<v,w>的意义或信息。

基本操作P：…

}ADT Graph

# 基本操作

结构的建立和销毁

对顶点的访问操作

插入或删除顶点

插入和删除弧

对邻接点的操作

遍历

# 结构的建立和销毁

```
CreatGraph(&G, V, VR):
// 按定义(V, VR) 构造图

DestroyGraph(&G):
// 销毁图
```

## 对顶点的访问操作

```
LocateVex(G, u);
    // 若G中存在顶点u, 则返回该顶点在
    // 图中“位置”; 否则返回其它信息。
GetVex(G, v);// 返回v 的值。
PutVex(&G, v, value); // 对v赋值value。
```

## 对邻接点的操作

```
FirstAdjVex(G, v);
  // 返回v 的“第一个邻接点”。若该顶点
  // 在G 中没有邻接点，则返回“空”。
NextAdjVex(G, v, w);
  // 返回v的(相对于w 的)“下一个邻接点”。
  // 若w是v的最后一个邻接点，则返回“空”。
```

## 插入或删除顶点

```
InsertVex(&G, v);
    // 在图G中增添新顶点v。
DeleteVex(&G, v);
    // 删除G中顶点v及其相关的弧。
```

# 插入和删除弧

```
InsertArc(&G, v, w);
  // 在G中增添弧<v,w>，若G是无向的，
  // 则还增添对称弧<w,v>。
DeleteArc(&G, v, w);
  // 在G中删除弧<v,w>，若G是无向的，
  // 则还删除对称弧<w,v>。
```

# 遍历

```
DFSTraverse(G, v, Visit());
  // 从顶点v起深度优先遍历图G, 并对每
  // 个顶点调用函数Visit一次且仅一次

BFSTraverse(G, v, Visit());
  // 从顶点v起广度优先遍历图G, 并对每
  // 个顶点调用函数Visit一次且仅一次。
```

# 7.2 图的存储结构

一、图的数组(邻接矩阵)存储表示

二、图的邻接表存储表示

三、有向图的十字链表存储表示

# 一、图的邻接矩阵存储表示

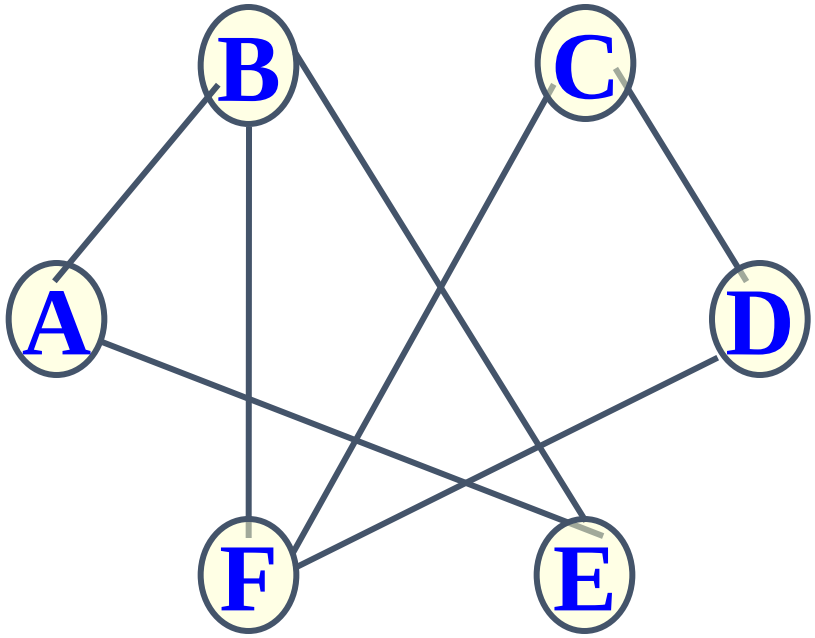
## 1. 图的邻接矩阵表示

设图 A = (V, E)是一个有 n 个顶点的图，

图的邻接矩阵是一个二维数组$A_{ij}$：

$$A_{ij}=\begin{cases} 0 & (i,j)\text{或}<i,j>\notin E \\ 1 & (i,j)\text{或}<i,j>\in E \end{cases}$$

# (1) 无向图的邻接矩阵表示

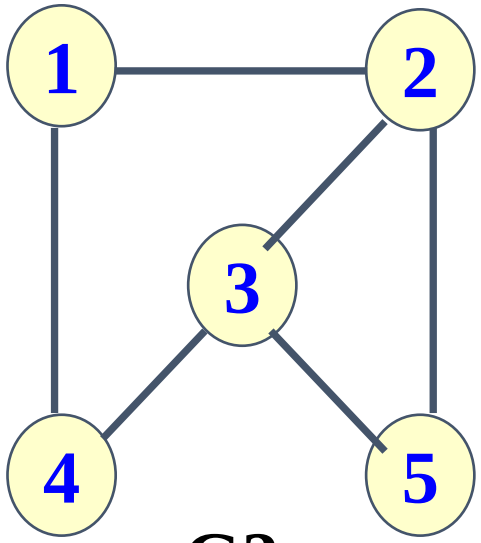



| | A | B | C | D | E | F |
|---|---|---|---|---|---|---|
| A | 0 | 1 | 0 | 0 | 1 | 0 |
| B | 1 | 0 | 0 | 0 | 1 | 1 |
| C | 0 | 0 | 0 | 1 | 0 | 1 |
| D | 0 | 0 | 1 | 0 | 0 | 1 |
| E | 1 | 1 | 0 | 0 | 0 | 0 |
| F | 0 | 1 | 1 | 1 | 0 | 0 |

注：无向图的邻接矩阵是对称的

在无向图中，统计第 i 行（列）1 的个数可得顶点i的度。

例




G2

| | ① | ② | ③ | ④ | ⑤ |
|---|---|---|---|---|---|
| ① | 0 | 1 | 0 | 1 | 0 |
| ② | 1 | 0 | 1 | 0 | 1 |
| ③ | 0 | 1 | 0 | 1 | 1 |
| ④ | 1 | 0 | 1 | 0 | 0 |
| ⑤ | 0 | 1 | 1 | 0 | 0 |

顶点2的度=3

## 从无向图的邻接矩阵可以得出如下结论：

（1）矩阵是对称的，可压缩存储（上/下）三角）；

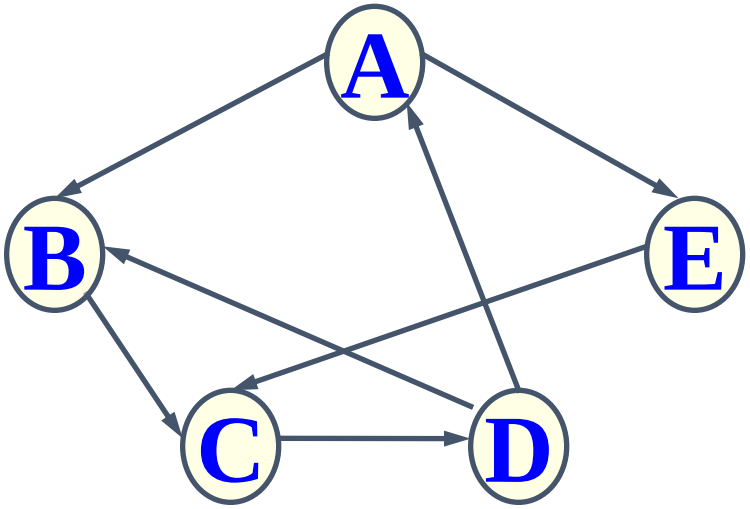
（2）顶点Vi 的度是第i行或第i 列中1的个数；

（3）矩阵中1的个数的一半为图中边的数目；

（4）很容易判断顶点Vi 和顶点Vj之间是否有边相连(看矩阵中i行j列值是否为1)；

（5）顶点不变，在图中增加（删除）边：只需对二维数组对应分量赋值1（清0）。

## (2) 有向图的邻接矩阵表示

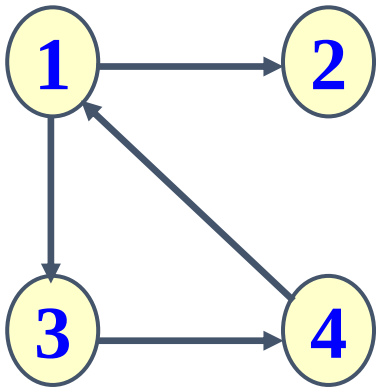



| | A | B | C | D | E |
|---|---|---|---|---|---|
| A | 0 | **1** | 0 | 0 | **1** |
| B | 0 | 0 | **1** | 0 | 0 |
| C | 0 | 0 | 0 | **1** | 0 |
| D | **1** | **1** | 0 | 0 | 0 |
| E | 0 | 0 | **1** | 0 | 0 |

注：有向图的邻接矩阵为非对称矩阵

在有向图中，统计第 i 行 1 的个数可得顶点 i 的出度，统计第 j 列 1 的个数可得顶点 j 的入度。

例




G1

| | ① | ② | ③ | ④ |
|---|---|---|---|---|
| ① | 0 | 1 | 1 | 0 |
| ② | 0 | 0 | 0 | 0 |
| ③ | 0 | 0 | 0 | 1 |
| ④ | 1 | 0 | 0 | 0 |

顶点2的出度=0；入度=1。

## 从有向图的邻接矩阵可以得出如下结论：

（1） 矩阵不一定对称；

（2） 第i行中1的个数为顶点Vi的出度；

（3） 第j列中1的个数为顶点Vj的入度；

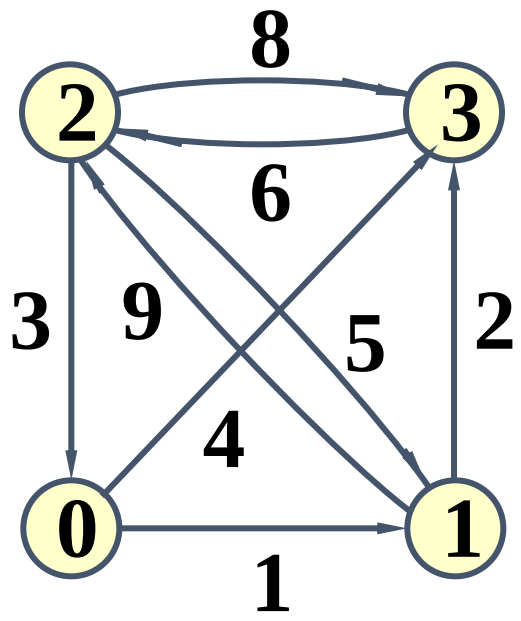
（4） 矩阵中1的个数为图中弧的数目；

（5） 很容易判断顶点Vi和顶点Vj是否有弧相连；

（6） 顶点不变，在图中增加（删除）边：只需对二维数组对应分量赋值1（清0）。

## 2. 网络的邻接矩阵

$$A[i,j]=\begin{cases}\omega_{ij}, \text{若}(v_i,v_j)\text{或}<v_i,v_j>\in E\\ \infty, \text{其它}\end{cases}$$




$$A[i,j]=\begin{bmatrix}\infty & 1 & \infty & 4\\ \infty & \infty & 9 & 2\\ 3 & 5 & \infty & 8\\ \infty & \infty & 6 & \infty\end{bmatrix}$$

# 图的邻接矩阵表示优缺点

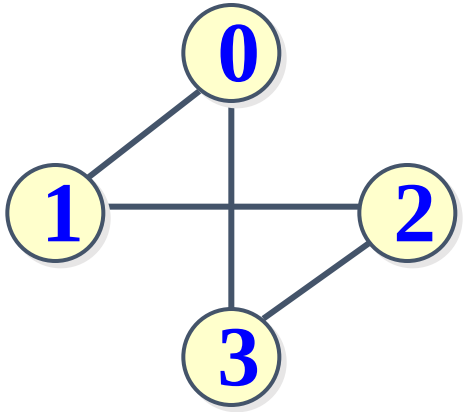
**优点：**

容易实现图的操作。如：求顶点的度、判断顶点之间是否有边（弧）、找顶点的邻接点等等。

**缺点：**

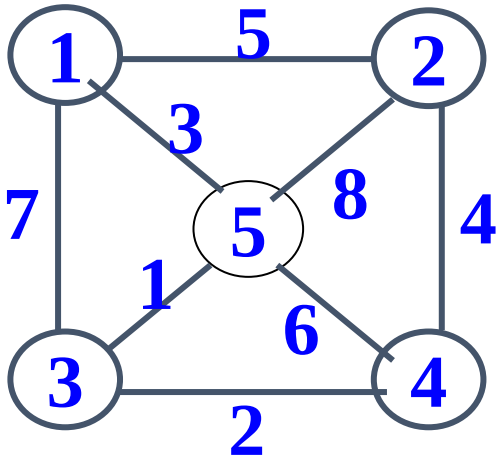
n个顶点需要*n***n*个单元存储边(弧)，对稀疏图而言尤其浪费空间。

# 写出下列图的邻接矩阵










$$\begin{bmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}$$

$$\begin{bmatrix} \infty & 5 & 7 & \infty & 3 \\ 5 & \infty & \infty & 4 & 8 \\ 7 & \infty & \infty & 2 & 1 \\ \infty & 4 & 2 & \infty & 6 \\ 3 & 8 & 1 & 6 & \infty \end{bmatrix}$$

# 3. 图的邻接矩阵类型描述

```
# define INFINITY 1000
#define MAX_VERTEX_NUM 20
typedef enum {DG,DN,UDG,UDN} GraphKind;
typedef struct ArcCell { // 边定义
    VRType  adj;    // VRType是顶点关系类型。
        // 对无权图，用1或0表示相邻否；
        // 对带权图（网络），则为权值类型。
    InfoType  *info;  // 该弧相关信息的指针
} ArcCell, AdjMatrix[MAX_VERTEX_NUM]
                    [MAX_VERTEX_NUM];
```

```
# define INFINITY 1000
#define MAX_VERTEX_NUM 20
typedef enum {DG,DN,UDG,UDN} GraphKind;
typedef struct ArcCell { // 边定义
    VRType  adj;   // VRType是顶点关系类型。
        // 对无权图，用1或0表示相邻否；对带权图，则为权值类型。
    InfoType  *info;  // 该弧相关信息的指针
} ArcCell, AdjMatrix[MAX_VERTEX_NUM][MAX_VERTEX_NUM];
```

```
typedef struct { // 图的定义
     VertexType   vexs[MAX_VERTEX_NUM];  // 顶点信息
     AdjMatrix    arcs;      // 弧的信息
    int    vexnum, arcnum;  // 顶点数，弧数
    GraphKind   kind;     // 图的种类标志
 } MGraph;
```

# 第九次作业：

## 实现图的FirstAdjVex基本操作：

```
int FirstAdjVex(MGraph G, int v)
```

## 二、图的邻接表存储表示

### 1. 图的邻接表表示

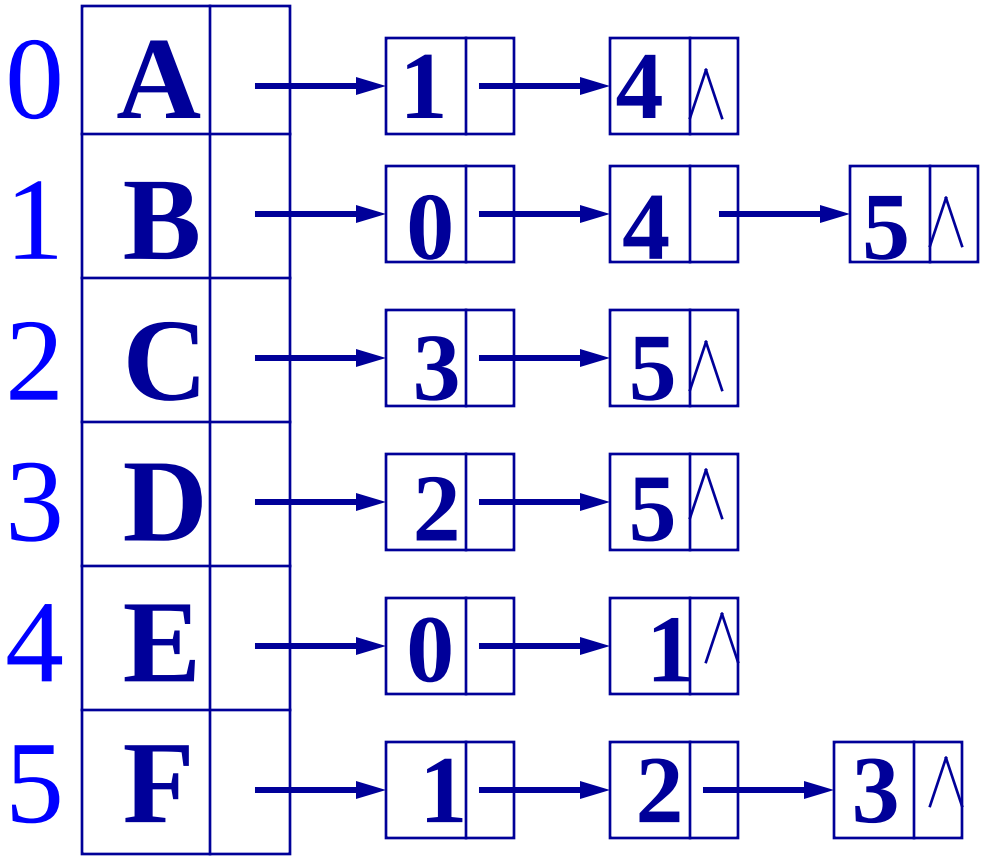
一种顺序分配与链式分配相结合的存储方法，它包括两部分：

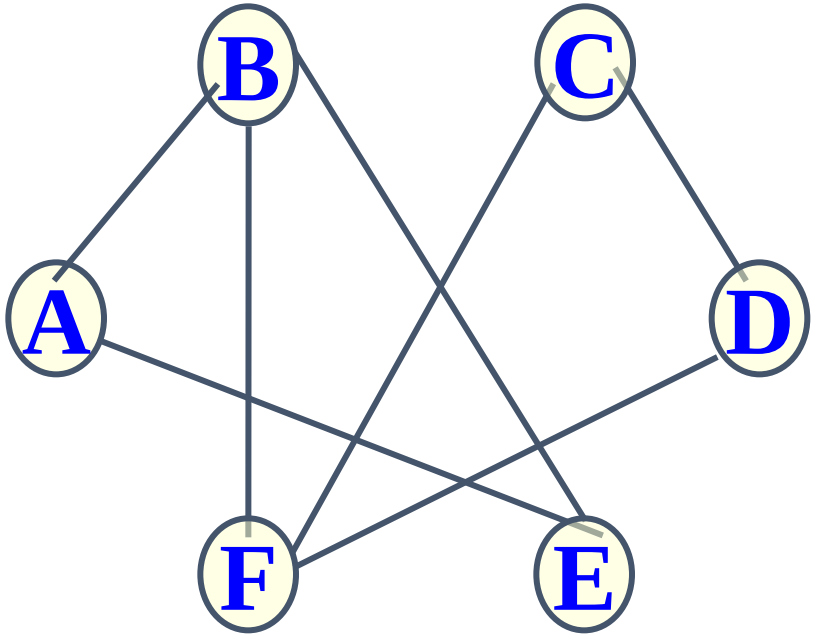
① 单链表：用来存放边/弧的信息；

② 数组：主要用来存放顶点本身的数据信息。

# (1) 无向图的邻接表

同一个顶点发出的边在同一个边链表中，每一个链结点代表一条边(边结点）。




无向图中顶点$V_i$的度为第i个单链表中的结点数。

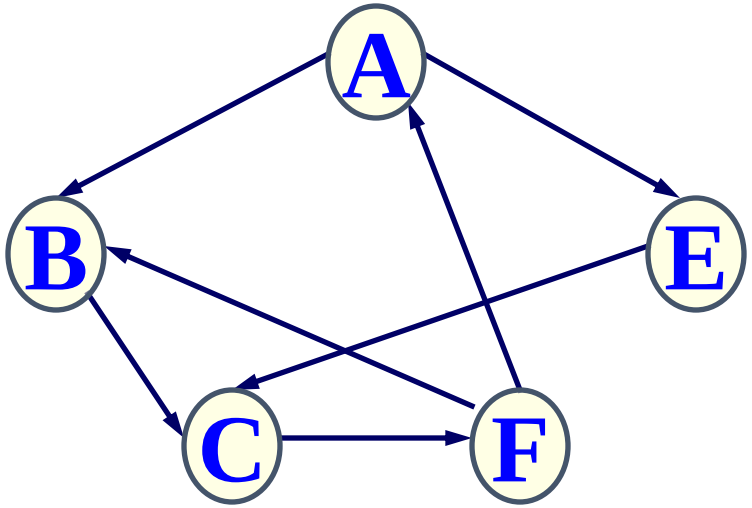
# 从无向图的邻接表可以得到如下结论：

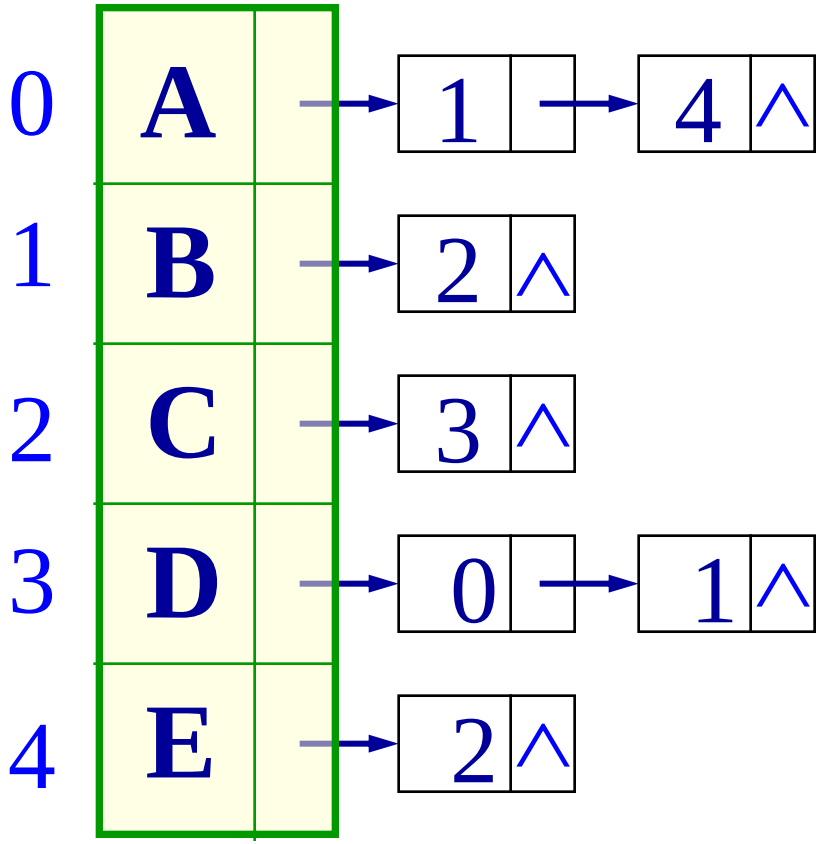
（1）第i个链表中边结点的数目为顶点Vi的度；

（2）所有链表中边结点数目的一半为图中边数；

（3）占用的存储单元数目为n+2e。

# (2) 有向图的邻接表




可见，在有向图的邻接表中不易找到指向该顶点的弧。

有向图中顶点$V_i$的出度为第i个单链表中的结点数。

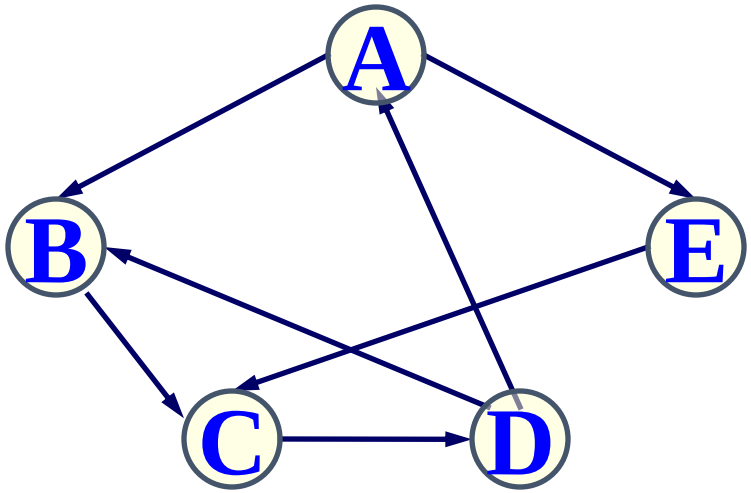
# 从有向图的邻接表可以得到如下结论：

（1）第i个链表中结点数目为顶点Vi的出度；但不能求出顶点的入度；

（2）所有链表中结点数目为图中弧数；

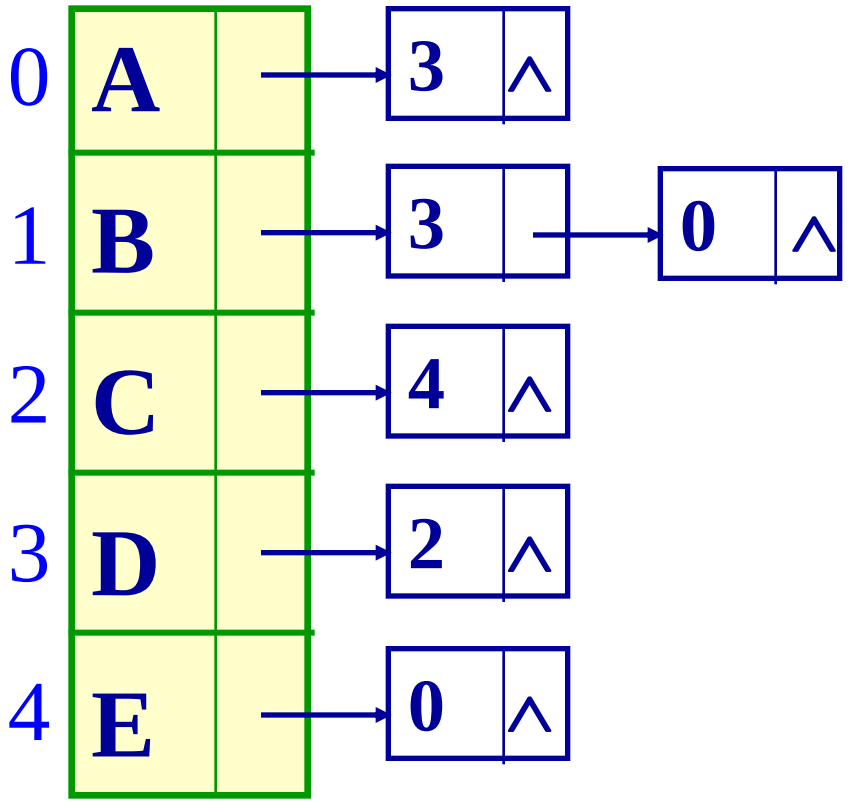
（3）占用的存储单元数目为n+e。

# (3) 有向图的逆邻接表（求顶点的入度）




在有向图的逆邻接表中，对每个顶点，链接的是指向该顶点的弧。

有向图中顶点$V_i$的入度为第i个单链表中的结点数。



## (4) 图的邻接表表示优缺点

### 邻接表的优点：

空间效率高；容易寻找顶点的邻接点；

### 邻接表的缺点：

判断两顶点间是否有边或弧，需搜索两结点对应的单链表，没有邻接矩阵方便。

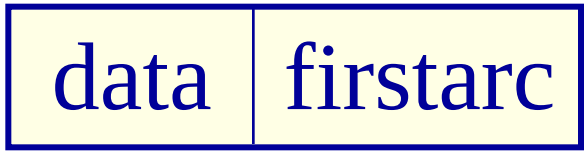
# 2. 图的邻接表类型描述

表结点结构




```
typedef struct ArcNode {   // 表结点定义
    int      adjvex;   //该弧所指向的顶点的位置
    struct ArcNode  *nextarc;
                          //指向下一条弧的指针
    InfoType   *info;   //该弧相关信息的指针
} ArcNode;
```

## 顶点的结点结构




```
typedef struct VNode { //表头结点
  VertexType  data;   // 顶点信息
  ArcNode  *firstarc;
                  // 指向第一条依附该顶点的弧
  } VNode, AdjList[MAX_VERTEX_NUM];
```

```
typedef struct ArcNode {//表结点定义
    int       adjvex;   //该弧所指向的顶点的位置
    struct ArcNode  *nextarc; //指向下一条弧的指针
    InfoType   *info;   //该弧相关信息的指针
} ArcNode;
```

```
typedef struct VNode { //表头结点
    VertexType  data;   // 顶点信息
    ArcNode  *firstarc;    // 指向第一条依附该顶点的弧
} VNode, AdjList[MAX_VERTEX_NUM];
```

```
typedef struct {  //图结构定义
    AdjList  vertices; // 图的顶点结构数组
    int   vexnum, arcnum;     int   kind; // 图的种类标志
} ALGraph;
```

# 第十次作业：

## 实现图的FirstAdjVex基本操作：

```
int FirstAdjVex(ALGraph G, int v)
```

# 讨论：邻接表与邻接矩阵有什么异同之处？

1. 联系

邻接表中每个链表对应于邻接矩阵中的一行；
链表中结点个数等于一行中非零元素的个数。

2. 区别

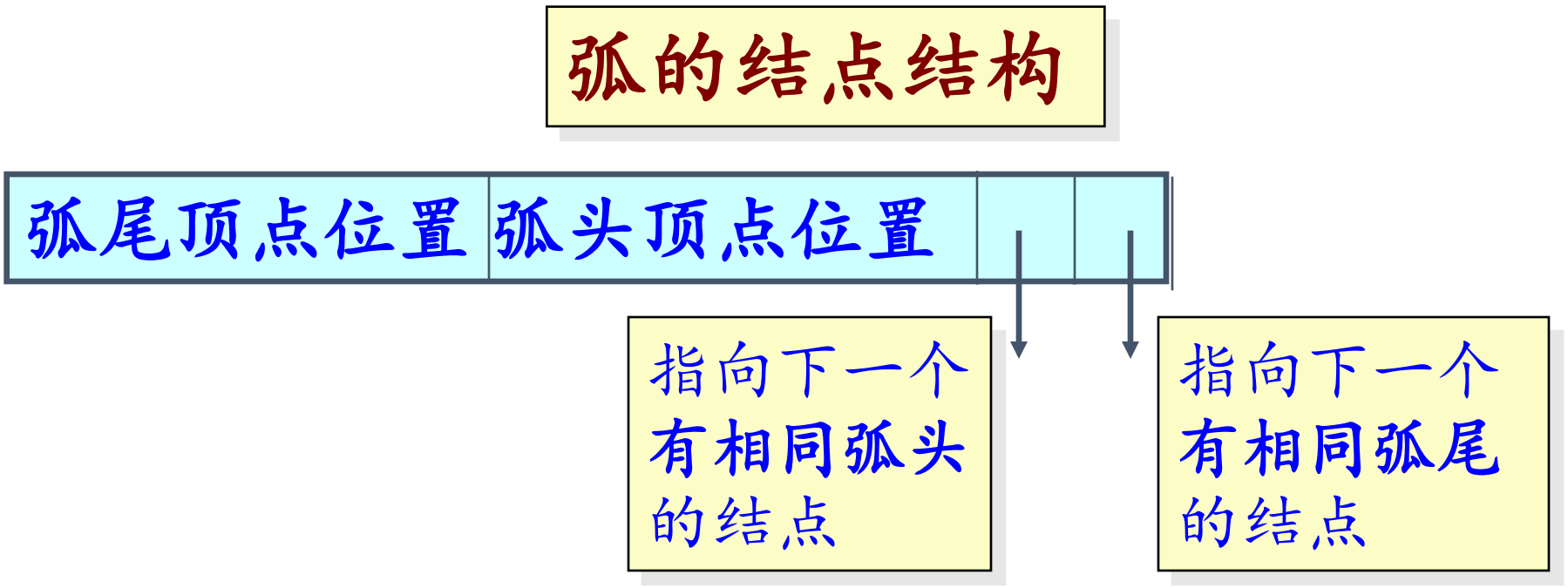
① 对于任一确定的无向图，邻接矩阵是唯一的（行列号与顶点编号一致），但邻接表不唯一（链接次序与顶点编号无关）。

② 邻接矩阵的空间复杂度为$O(n^2)$，而邻接表的空间复杂度为$O(n+e)$。

3. 用途

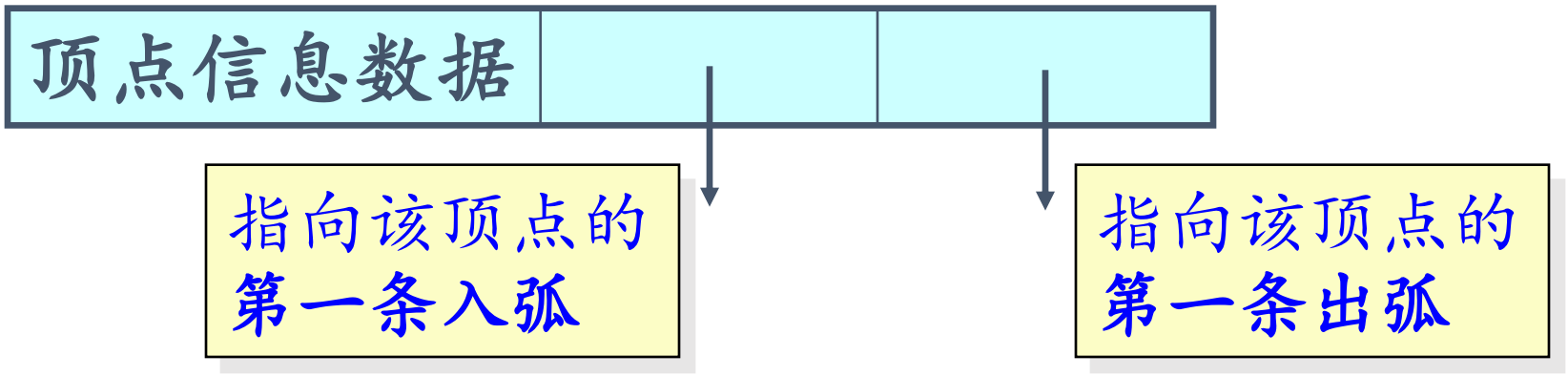
邻接矩阵多用于稠密图的存储；
而邻接表多用于稀疏图的存储。

# 三、有向图的十字链表存储表示



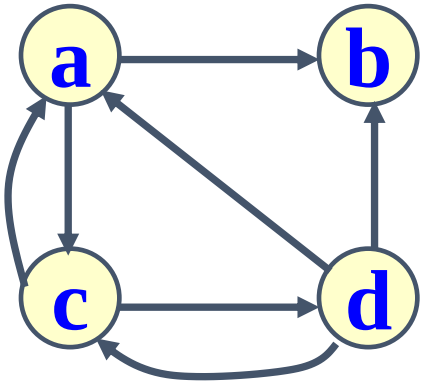

```
typedef struct ArcBox { // 弧结点结构表示
    int tailvex, headvex;
    struct ArcBox  *hlink, *tlink;
} VexNode;
```

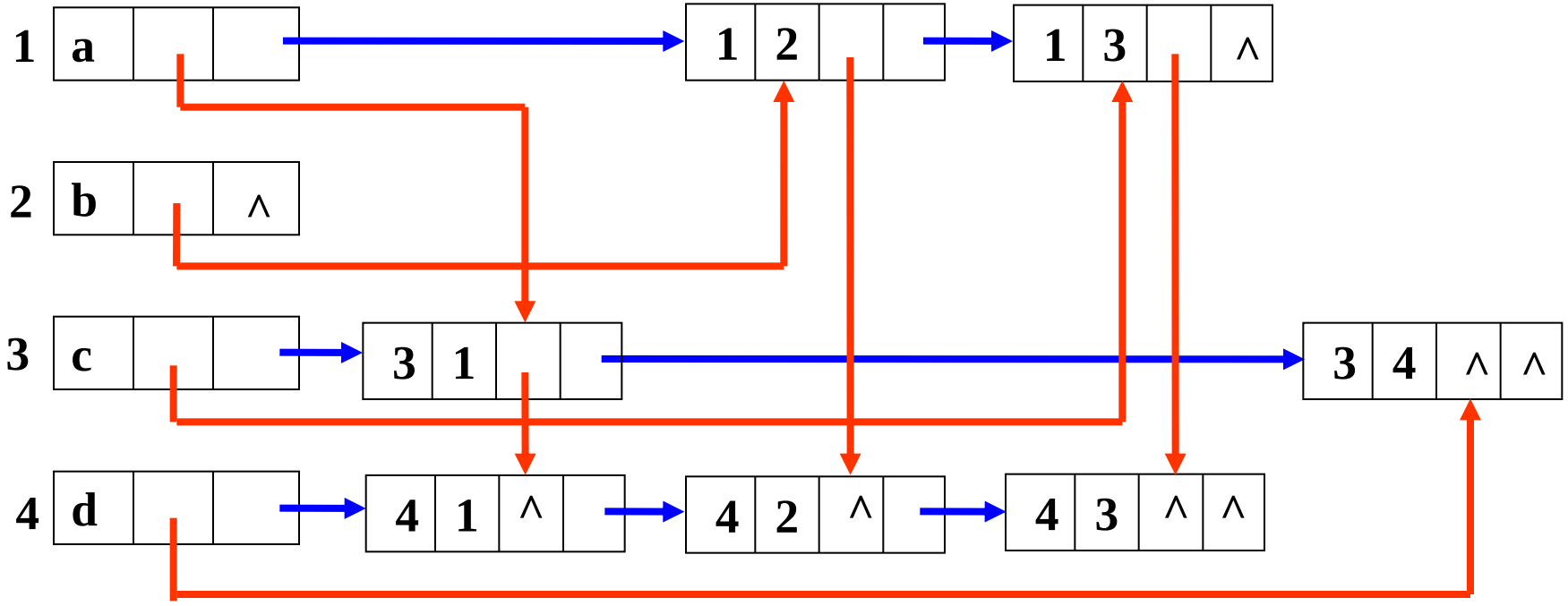
## 顶点的结点结构




```
typedef struct VexNode { // 顶点的结构表示
    VertexType  data;
    ArcBox  *firstin, *firstout;
} VexNode;
```

例



```
typedef struct ArcBox { // 弧结点结构表示
    int tailvex, headvex;   InfoType  *info;
    struct ArcBox  *hlink, *tlink;
} VexNode;
```

```
typedef struct VexNode { // 顶点的结构表示
     VertexType  data;
     ArcBox  *firstin, *firstout;
} VexNode;
```

```
typedef struct { // 有向图的十字链表
    VexNode  xlist[MAX_VERTEX_NUM];    // 顶点结点
    int   vexnum, arcnum; //有向图的当前顶点数和弧数
} OLGraph;
```

# 7.3 图的遍历

与树结构类似：从图中某一顶点出发访遍图中所有的顶点，且使每个顶点仅被访问一次，这一过程就叫做图的遍历 。

在图中，访问部分顶点后，可能又沿着其他边回到已被访问过的顶点。为保证每一个顶点只被访问一次，必须对顶点进行标记，一般用一个辅助数组 visit[MAX]作为对顶点的标记。

- 当顶点$v_i$未被访问，visit[i]值为0；
- 当$v_i$已被访问，则visit[i]值为1。

根据搜索路径的方向不同，图的遍历有两种方法：

- 深度优先搜索
- 广度优先搜索

它们对无向图和有向图都适用。

# 一、深度优先搜索 DFS
# ( Depth First Search )

## 1. 深度优先搜索的思想

深度优先搜索遍历类似于树的先序遍历。

从图中某个顶点$V_0$出发，访问此顶点，然后依次从$V_0$的各个未被访问的邻接点出发深度优先搜索遍历图，直至图中所有和$V_0$有路径相通的顶点都被访问到。

假定给定图G的初态是所有顶点均未被访问过，在G中任选一个顶点$V_i$作为遍历的初始点，则DFS可定义如下：

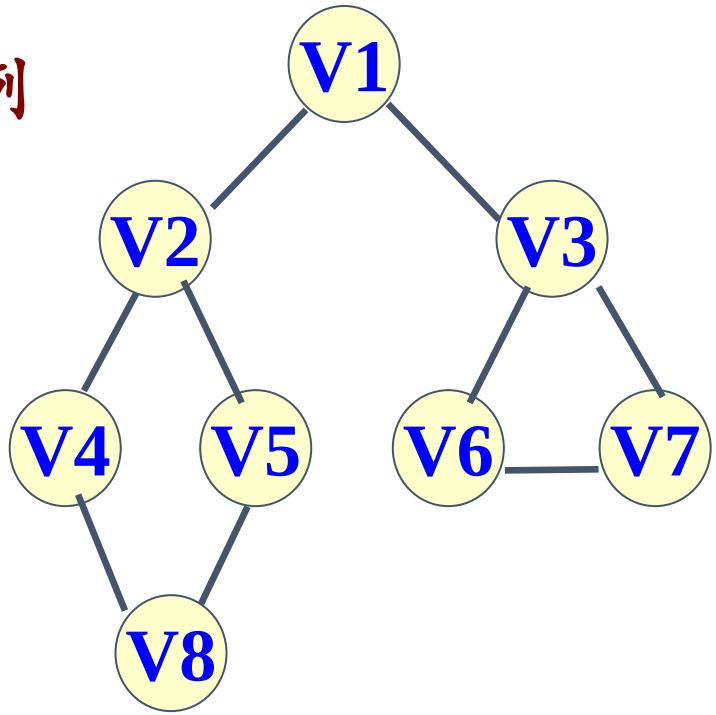
(1)首先访问顶点$V_i$，并将其访问标记置为访问过，即visited[i]=1;

(2) 然后搜索与顶点$V_i$有边相连的下一个顶点$V_j$

- 若$V_j$未被访问过，则访问它，并将其访问标记置为访问过，visited[j]=1，然后从$V_j$开始重复此过程；
- 若$V_j$已访问，再看与$V_i$有边相连的其它顶点；

(3)若与$V_i$有边相连的顶点都被访问过，则退回到前一个访问顶点并重复刚才过程，直到图中所有顶点都被访问完为止。

例




注意：由于没有规定访问邻接点的顺序，深度优先序列不是唯一的。

其中两种DFS序列：

(1) V1⇒ V2 ⇒V4 ⇒ V8 ⇒V5 ⇒V3 ⇒V6 ⇒V7

(2) V1⇒ V3 ⇒V7 ⇒ V6 ⇒V2 ⇒V4 ⇒V8 ⇒V5

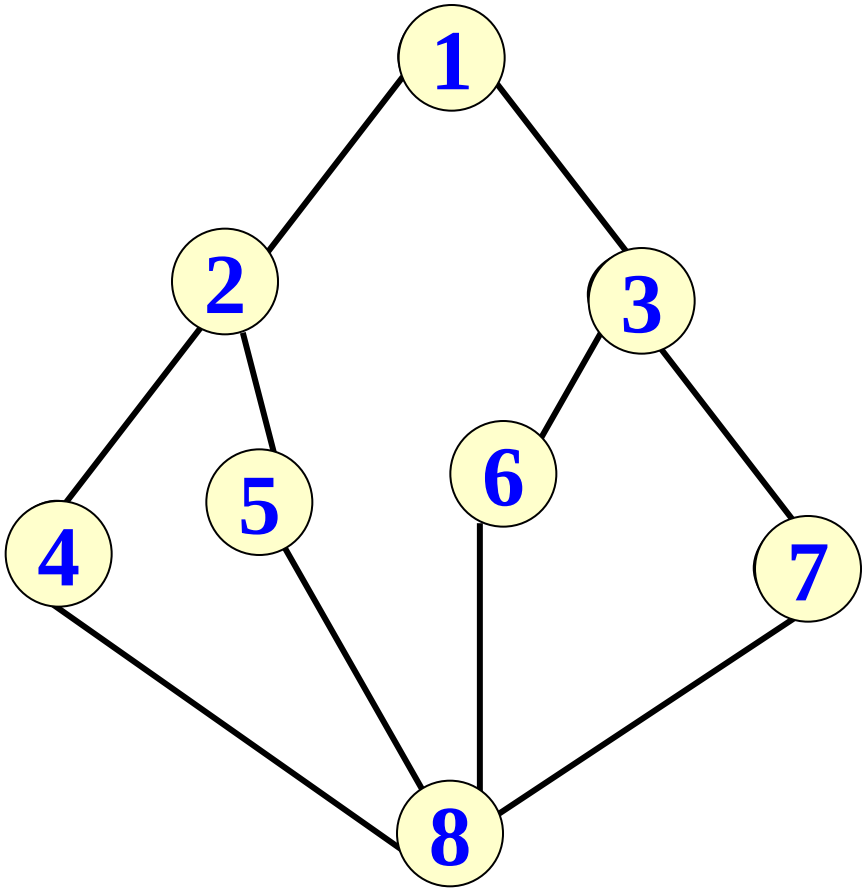
遍历图的过程实质上是对每个顶点查找其邻接点的过程。

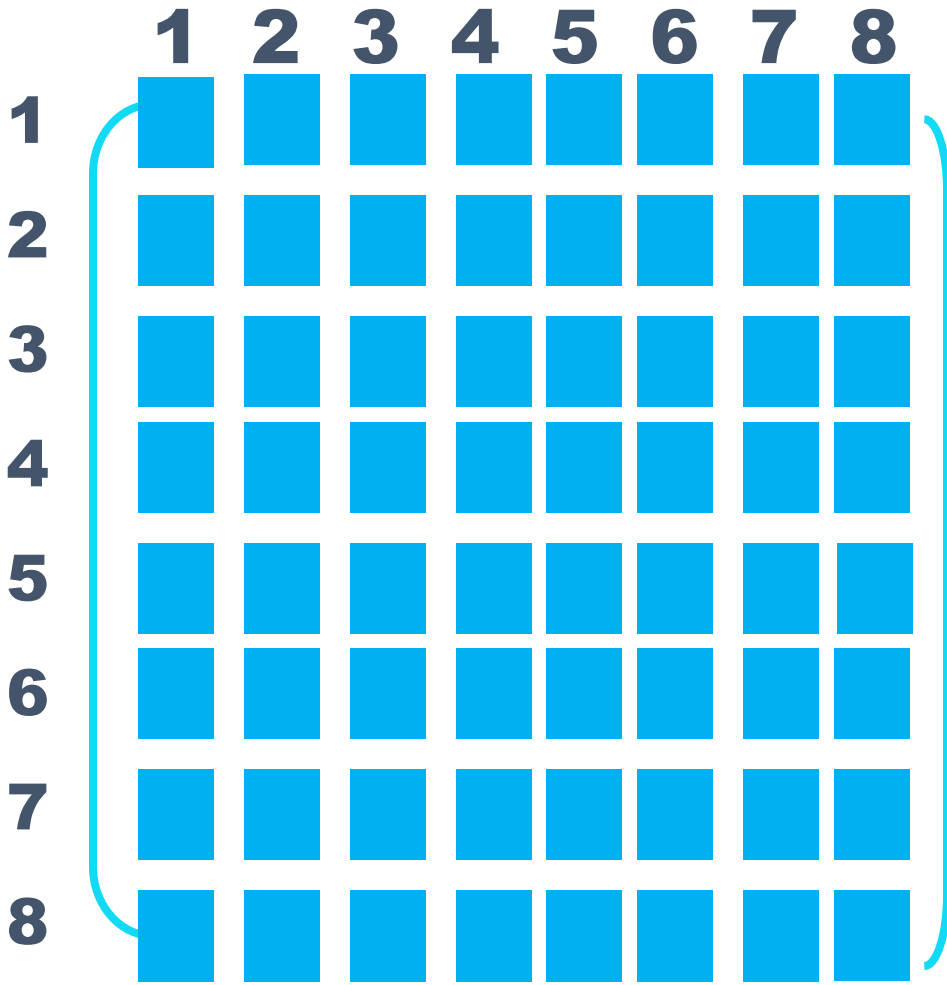
若图是连通的或强连通的，则从图中某一个顶点出发可以访问到图中所有顶点，否则只能访问到一部分顶点。

从某一个顶点出发的遍历结果是不唯一的。但若给定图的存储结构，则从某一顶点出发的遍历结果应是唯一的。

# (1) 用邻接矩阵实现图的深度优先搜索

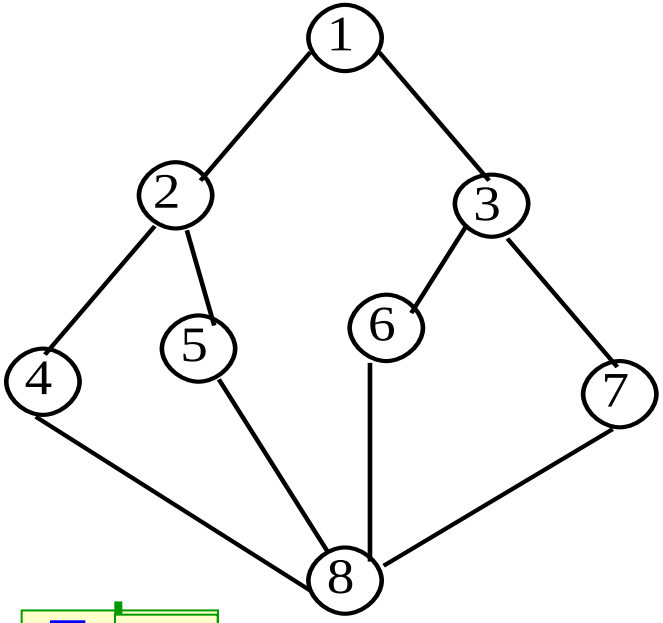
假设起始顶点为1

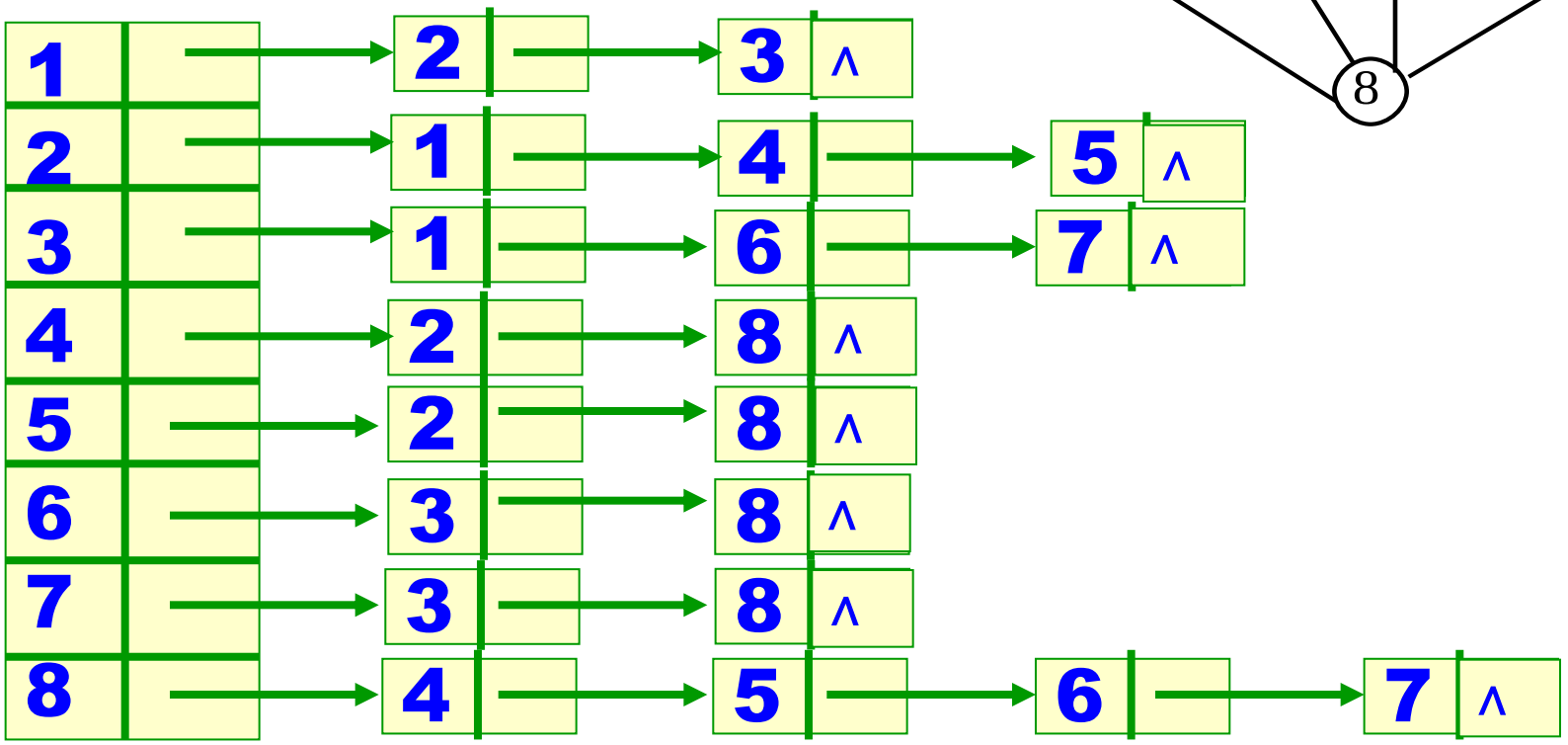


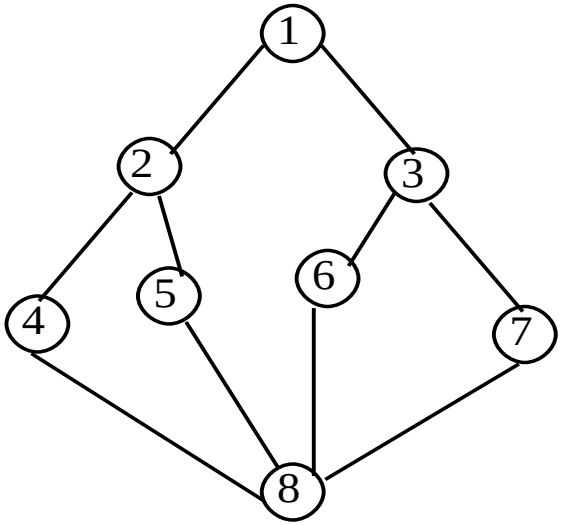



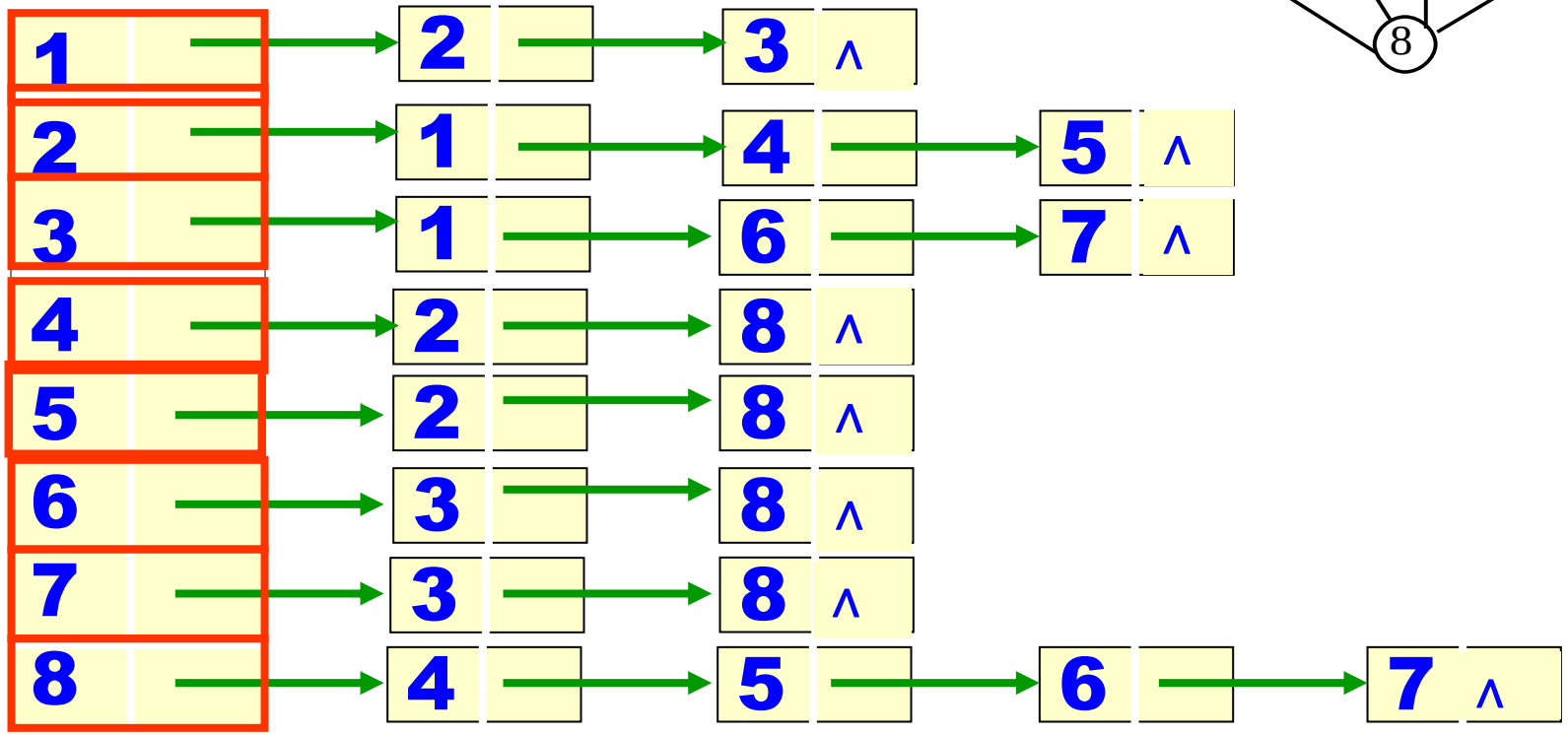

从顶点1的遍历结果为 1，2，4，8，5，6，3，7

# (2) 用邻接表实现图的深度优先搜索




1
2
3
4
5
6
7
8


1
2
3 ∧
2
1
4
5 ∧
3
1
6
7 ∧
4
2
8 ∧
5
2
8 ∧
6
3
8 ∧
7
3
8 ∧
8
4
5
6
7 ∧

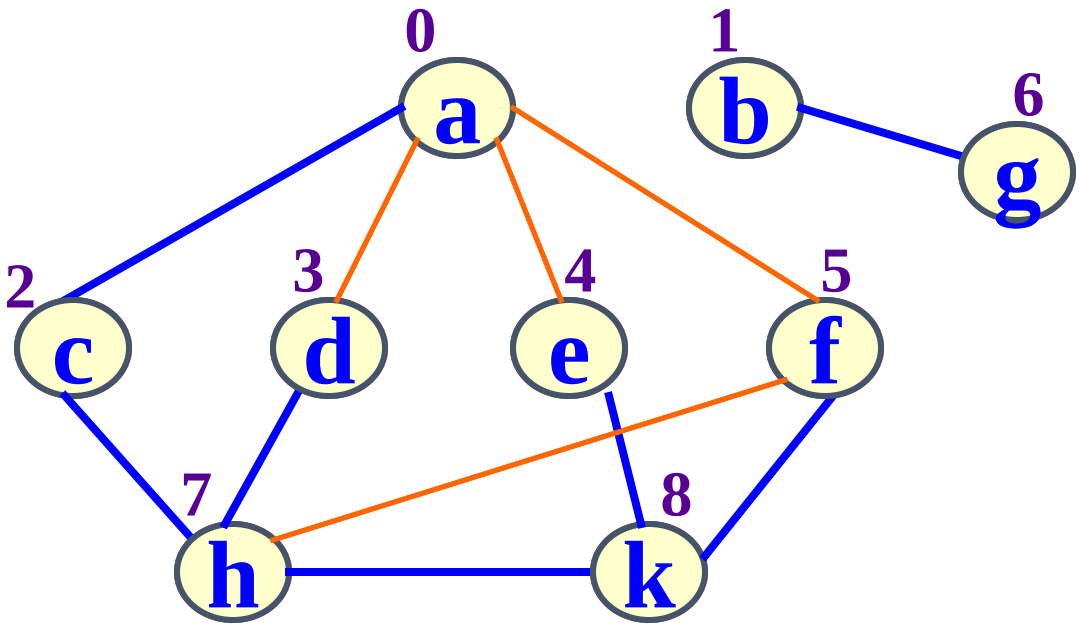
```
void DFS(Graph G, int v) {
    //从顶点v出发，深度优先搜索遍历连通图G
  visited[v] = TRUE;
  VisitFunc(v); // 访问第v个结点
  for(w=FirstAdjVex(G, v);
            w>=0; w=NextAdjVex(G,v,w))
      if (!visited[w]) DFS(G, w);
          // 对v的尚未访问的邻接顶点w
          // 递归调用DFS
} // DFS
```

## 3. 非连通图的深度优先搜索遍历

若图是非连通的或非强连通图，则从图中某一个顶点出发。不能用深度优先搜索访问到图中所有顶点，而只能访问到一个连通子图（连通分量）或只能访问到一个强连通子图（强连通分量）。

可在每个连通分量或强连通分量中选一个起始顶点，进行深度优先搜索遍历，最后将每个连通分量或强连通分量的遍历结果合起来，则得到整个非连通图的遍历结果。

例如：




| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|---|---|---|---|---|---|---|---|---|---|
| 访问标志： | T | T | T | T | T | T | T | T | T |

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| 访问次序： | a | c | h | d | k | f | e | b | g |

```
void DFSTraverse(Graph G,
                 Status(*Visit)(intv)){
                 // 对图G 作深度优先遍历。
  VisitFunc= Visit;
    //使用全局变量，使DFS不必设函数指针参数
  for(v=0; v<G.vexnum; ++v)
    visited[v] = FALSE; //访问标志数组初始化
  for(v=0; v<G.vexnum; ++v)
    if(!visited[v]) DFS(G, v);
        // 对尚未访问的顶点调用DFS
}
```

# 二、广度优先搜索遍历图

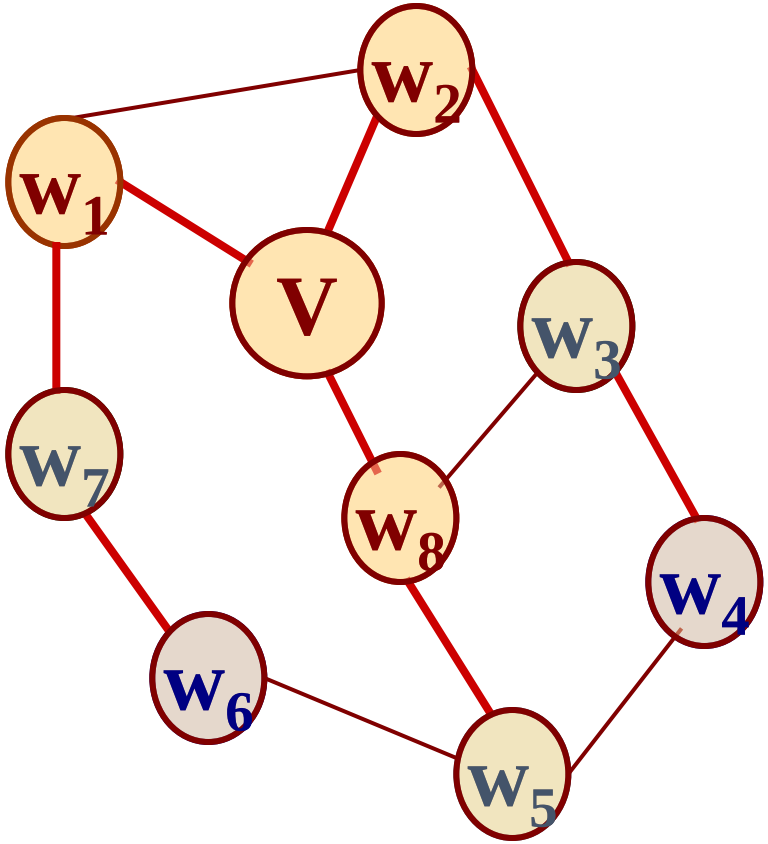
## 1. 广度优先搜索思想　　类似于树的层次遍历

(1)首先访问顶点Vi，并将其访问标志置为已被访问，即visited[i]=1;

(2)接着依次访问与顶点Vi有边相连的所有顶点W1，W2，…，Wt;

(3)然后再按顺序访问与W1，W2，…，Wt有边相连又未曾访问过的顶点;

依此类推，直到图中所有顶点都被访问完为止。




由于没有规定访问邻接点的顺序，广度优先序列不是唯一的。

- V⇒W1 ⇒ W2 ⇒ W8 ⇒W7 ⇒ W3 ⇒W5 ⇒W6 ⇒W4
- V⇒W2 ⇒ W1 ⇒ W8 ⇒W3 ⇒ W7 ⇒W5 ⇒W6 ⇒W4

## 3. 非连通图的广度优先搜索

在每个连通分量或每个强连通分量中都选一个顶点，进行广度优先搜索遍历，最后将每个连通分量或每个强连通分量的遍历结果合起来，则得到整个非连通图或非强连通图的广度优先搜索遍历序列。

```
void BFSTraverse(Graph G,
                 Status (*Visit)(int v)){
  for(v=0; v<G.vexnum; ++v)
     visited[v] = FALSE; //初始化访问标志
  InitQueue(Q);// 置空的辅助队列Q
  for( v=0; v<G.vexnum; ++v )
     if (!visited[v]) { //v尚未访问
        ......
     } //if
}// BFSTraverse
```

```
visited[v] = TRUE; Visit(v); // 访问v
EnQueue(Q, v); // v入队列
while(!QueueEmpty(Q)) {
    DeQueue(Q, u); // 队头元素出队并置为u
    for(w=FirstAdjVex(G, u); w!=0;
                        w=NextAdjVex(G,u,w))
        if (!visited[w]){
            visited[w]=TRUE; Visit(w);
            EnQueue(Q, w); //访问的顶点w入队列
        } // if
} // while
```

# 7.4 图的连通性问题

- 生成树
  - 最小生成树

# 一、生成树

① 生成树（极小连通子图）：含有连通图中全部n个顶点，但只有n-1条边，且n-1条边不能构成回路。

② 遍历生成树 --- 特例（遍历过程中生成）

深度优先生成树与广度优先生成树。

③ 生成森林：非连通图每个连通分量的生成树一起组成非连通图的生成森林。

# 生成树的说明：

一个图可以有许多棵不同的生成树，

所有生成树具有以下共同特点：

① 生成树的顶点个数与图的顶点个数相同；

② 生成树是图的极小连通子图；

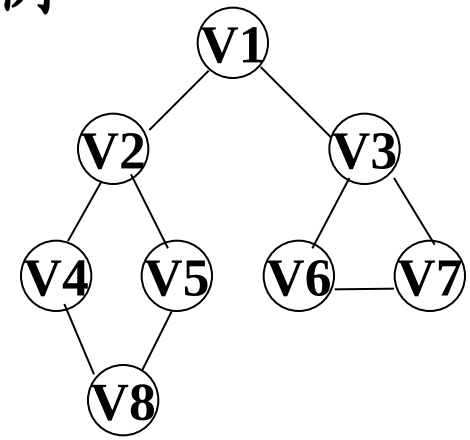
③ 一个有n个顶点的连通图的生成树有n-1条边；

④ 生成树中任意两个顶点间的路径是唯一的；

⑤ 在生成树中再加一条边必然形成回路；

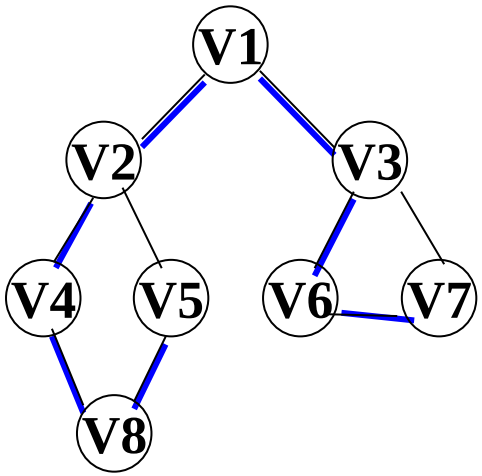
⑥ 含n个顶点n-1条边的图不一定是生成树；

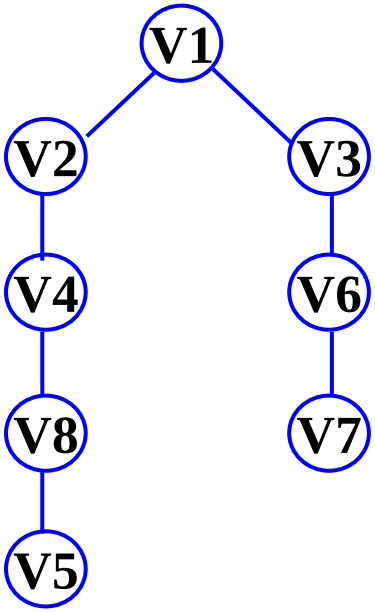
例



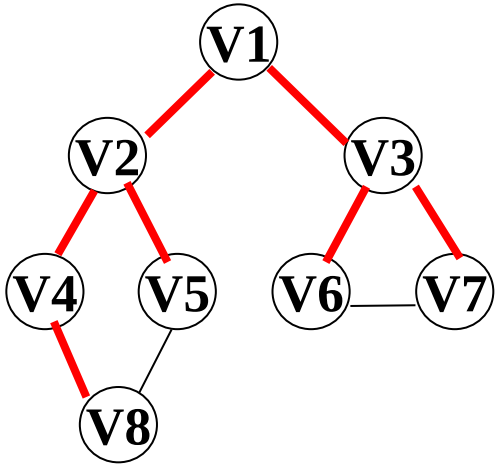

深度遍历：V1⇒ V2 ⇒V4 ⇒ V8 ⇒V5 ⇒V3 ⇒V6 ⇒V7

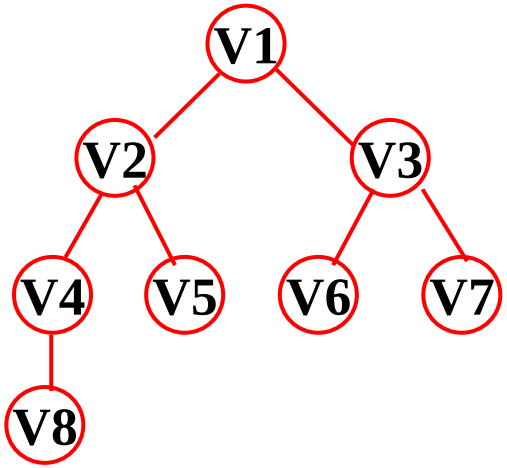
广度遍历：V1⇒ V2 ⇒V3 ⇒ V4 ⇒V5 ⇒V6 ⇒V7 ⇒V8



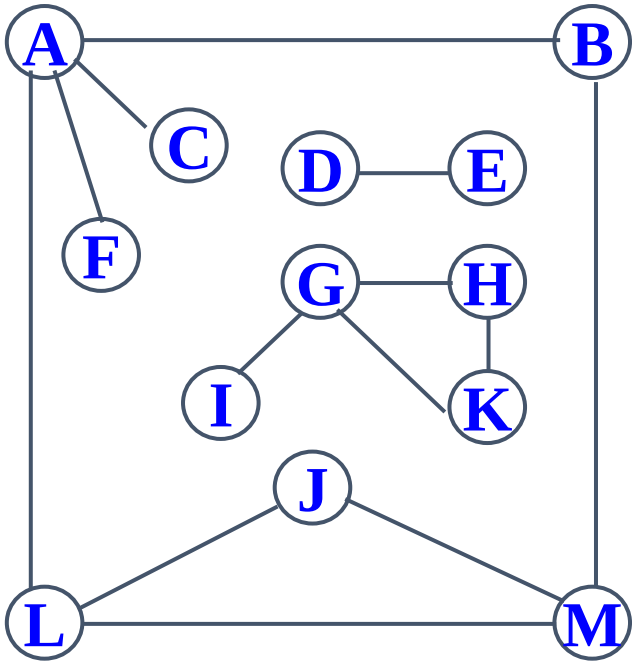

深度优先生成树

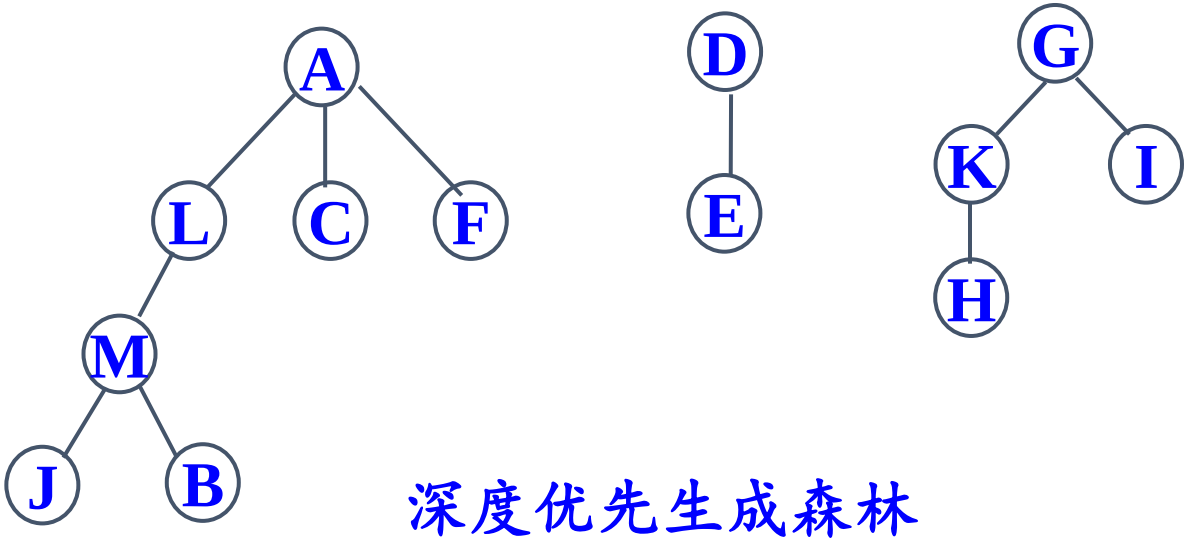



广度优先生成树

例




深度优先生成森林

# 二、最小生成树

1. 定义：若生成树中每条边上权值之和达到最小，称为最小生成树。

**注意：**

- 使用不同的遍历图的方法，可得到不同的生成树；
- 从不同的顶点出发，也可能得到不同的生成树。

**目标：**

在网的多个生成树中，寻找一个各边权值之和最小的生成树。

**构造最小生成树的基本准则：**

- 必须只使用该网中的边来构造最小生成树；
- 必须使用且仅使用$n-1$条边来连接网中的$n$个顶点；
- 不能使用产生回路的边。

## 2. 最小生成树的典型用途（城市公交网）

[问题描述]

有一张城市地图，图中的顶点为城市，无向边代表两个城市间的连通关系，边上的权为在这两个城市之间修建高速公路的造价，研究后发现，这个地图有一个特点，即任一对城市都是连通的。现在的问题是，要修建若干高速公路把所有城市联系起来，问如何设计可使得工程的总造价最少。

[输入]

n（城市数，1<=n<=100）；　　e（边数）；

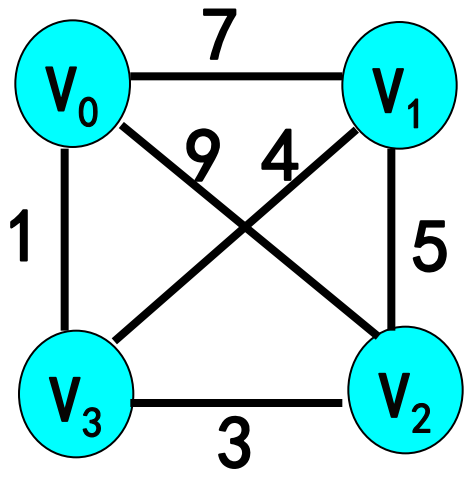
[输出]

工程的总造价最少

# 最小生成树的典型用途（城市公交网）

以一个4个城市的地图为例：

造价最高！！！




完全图：n个顶点，n(n-1)/2条边

生成树！！！n个顶点，n-1条边

寻找：

（1）满足任一对城市都是连通的，且边数达到最少的一个连通图。

（2）工程造价最少。

最小生成树！！！

n-1条边的权之和最小

# 3. 最小生成树算法

算法一：（普里姆算法）

算法二：（克鲁斯卡尔算法）

# (1) 普里姆算法

## 算法思想：

① 在图中任取一个顶点k作为开始点。

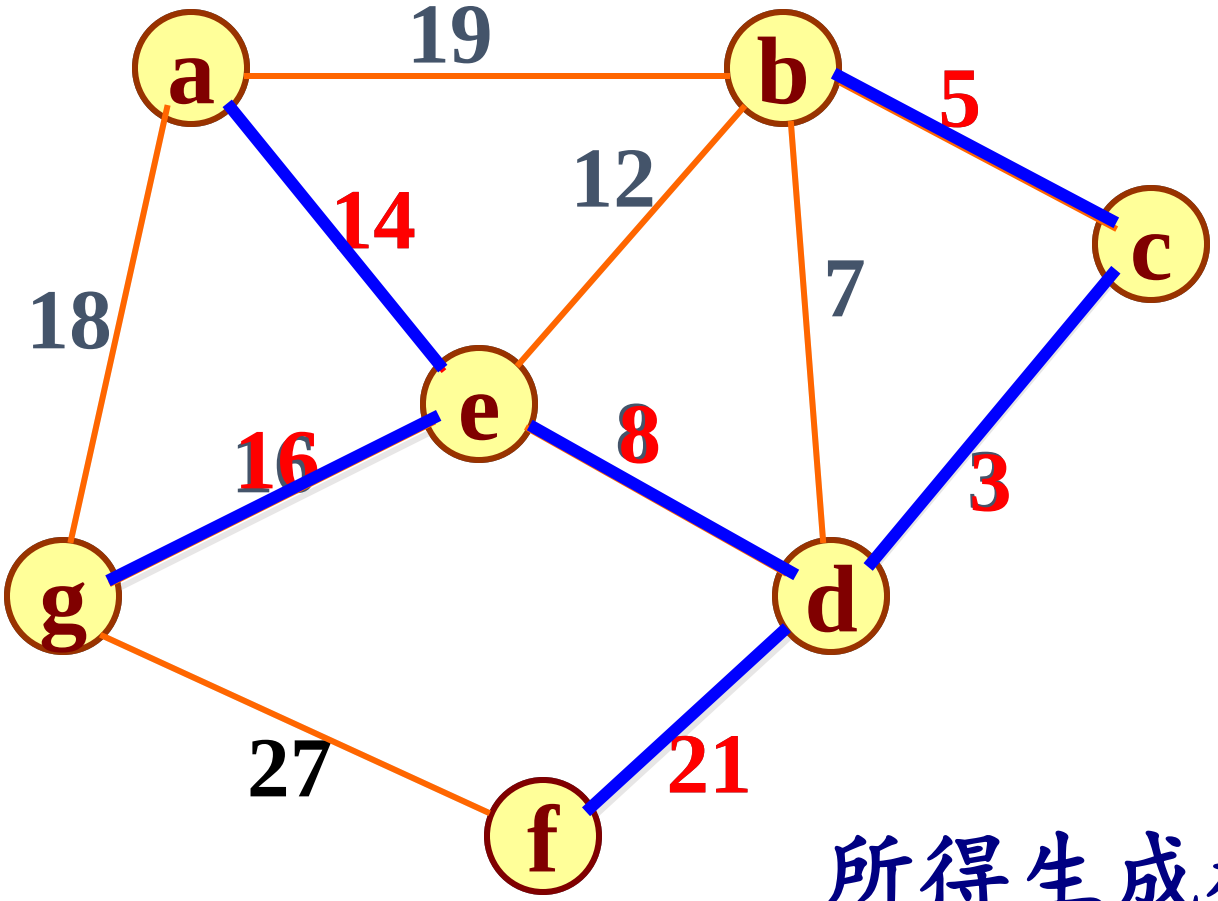
令U={k}，W=V-U，其中V为图中所有顶点集

② 在U中任意选取一个顶点，W中选取另一个顶点，使二者对应的边是最短的一条。

将该边作为最小生成树的边保存起来，并将该边顶点加入U集合中，并从W中删去对应的顶点。

③ 重复步骤②，直到W为空集止。

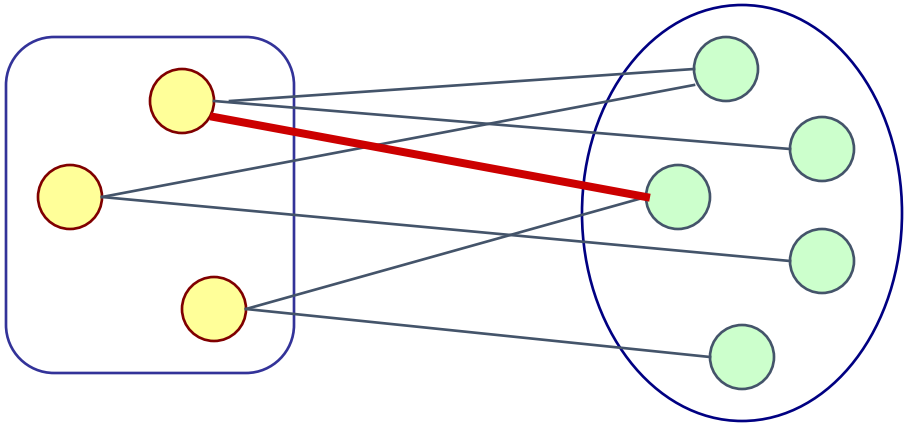
例如：

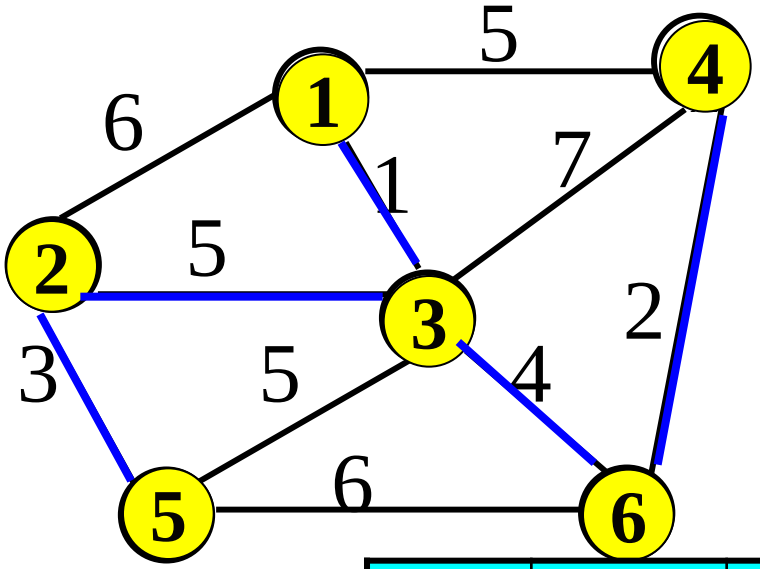



所得生成树权值和

= 14+8+3+5+16+21 = 67

一般情况下所添加的顶点应满足下列条件：在生成树的构造过程中，图中 n 个顶点分属两个集合：已落在生成树上的顶点集 U 和尚未落在生成树上的顶点集V-U，则应在所有连通U中顶点和V-U中顶点的边中选取权值最小的边。






U={1}, W={2,3,4,5,6}

| (1,2) | (1,3) | (1,4) | (1,5) | (1,6) |
|---|---|---|---|---|
| 6 | 1 | 5 | ∞ | ∞ |

U={1,3}, W={2,4,5,6}

| (1,2) | (1,4) | (1,5) | (1,6) | (3,2) | (3,4) | (3,5) | (3,6) |
|---|---|---|---|---|---|---|---|
| 6 | 5 | ∞ | ∞ | 5 | 7 | 5 | 4 |

U={1,3,6}, W={2,4,5}

| (1,2) | (1,4) | (1,5) | (3,2) | (3,4) | (3,5) | (6,2) | (6,4) | (6,5) |
|---|---|---|---|---|---|---|---|---|
| 6 | 5 | ∞ | 5 | 7 | 5 | ∞ | 2 | 6 |

U={1,3,6,4}, W={2,5}

| (1,2) | (1,5) | (3,2) | (3,5) | (6,2) | (6,5) | (4,2) | (4,5) |
|---|---|---|---|---|---|---|---|
| 6 | ∞ | 5 | 5 | ∞ | 6 | ∞ | ∞ |

U={1,3,6,4,2}, W={5}

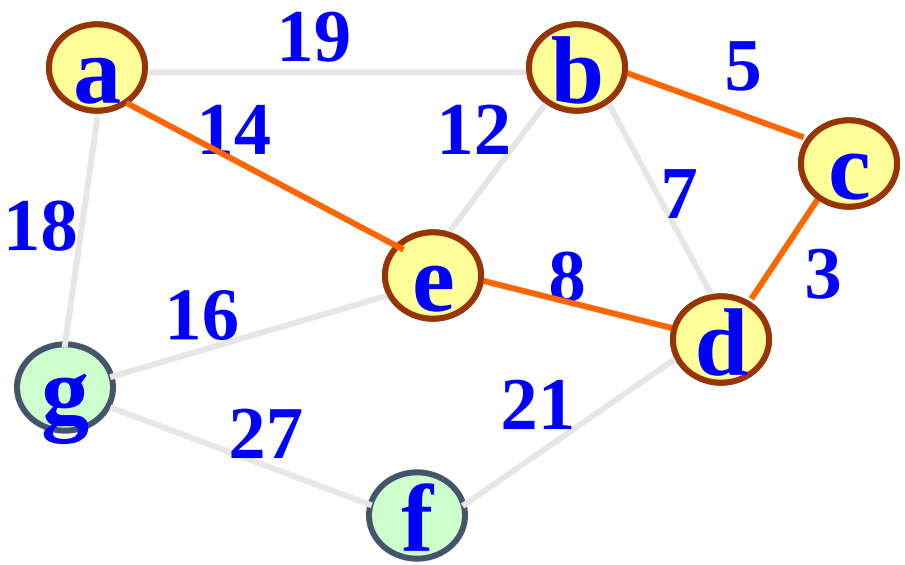
| (1,5) | (3,5) | (6,5) | (4,5) | (2,5) |
|---|---|---|---|---|
| ∞ | 5 | 6 | ∞ | 3 |

U={1,3,6,4,2,5}, W={}

在算法中需要设置一个辅助数组，对当前V—U集中的每个顶点，记录和顶点集U中顶点相连接的代价最小的边 。

```
struct {
    VertexType  adjvex;  // U集中的顶点
    VRType      lowcost;  // 边的权值
} closedge[MAX_VERTEX_NUM];
```

例如：




| i / closedge | 0 | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|---|
| Adjvex | | c | d | e | a | d | e |
| Lowcost | | 5 | 3 | 8 | 14 | 21 | 16 |

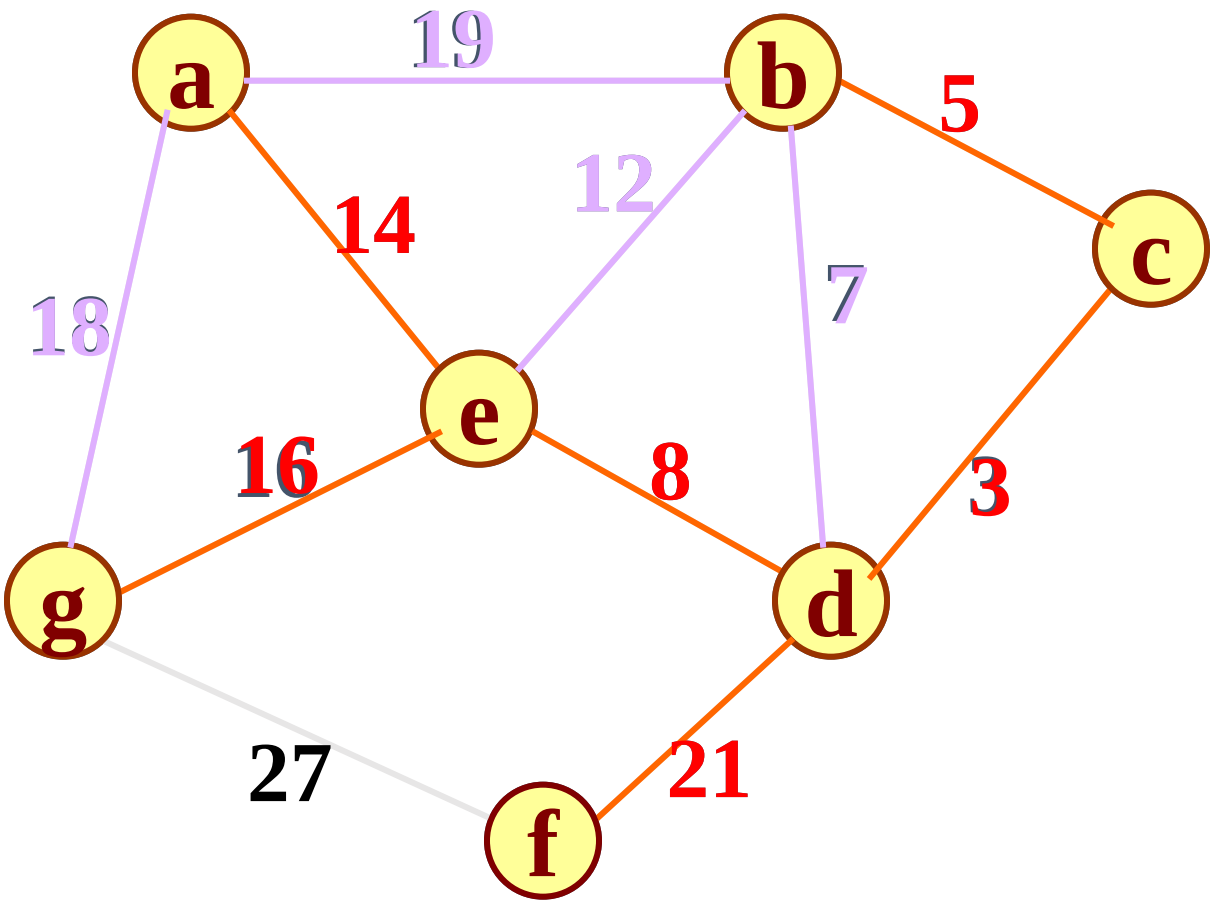
# （2）克鲁斯卡尔算法

算法思想：

① 将图中所有边按权值递增顺序排列；

② 依次选定取权值较小的边，但要求后面选取的边不能与前面选取的边构成回路，若构成回路，则放弃该条边，再去选后面权值较大的边，n个顶点的图中，选够n-1条边即可。

例如：



# 克鲁斯卡尔算法描述:

构造非连通图 ST=( V,{ } );

k = i = 0;　// k 计选中的边数

while (k<n-1) {

　++i;

　检查边集 E 中第 i 条权值最小的边(u,v);

　若(u,v)加入ST后不使ST中产生回路,

　则　输出边(u,v); 且　k++;

}

# 比较两种算法

| 算法名 | 普里姆算法 | 克鲁斯卡尔算法 |
| --- | --- | --- |
| 时间复杂度 | $O(n^2)$ | O(eloge) |
| 适应范围 | 稠密图 | 稀疏图 |

1. 熟悉图的各种存储结构及其构造算法，了解实际问题的求解效率与采用何种存储结构和算法有密切联系。

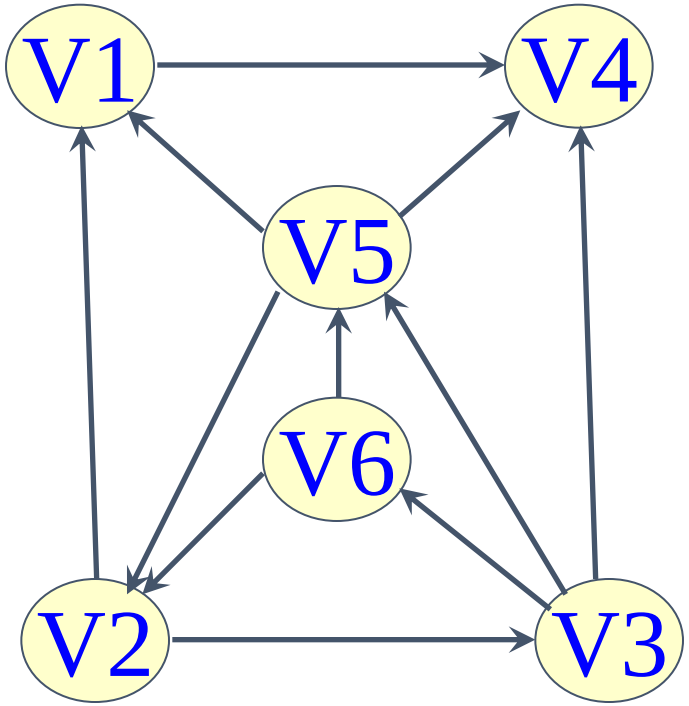
2. 熟练掌握图的两种搜索路径的遍历：遍历的逻辑定义、深度优先搜索和广度优先搜索的算法。

3. 理解最小生成树算法及其应用。

# 练 习

1. 画出下图的邻接矩阵、邻接表和十字链表。

2. 给出深度、广度遍历序列。



# 练 习

3. 按prim算法，从顶点V1出发生成最小生成树，按生成次序写出各条边。

4. 按kruskal算法生成最小生成树，按生成次序写出各条边。

求出最小生成树的权值

